



۱۳۰۷

دانشگاه صنعتی خواجه نصیرالدین طوسی

دانشکده مهندسی برق

گروه مهندسی برق

گزارش کارآموزی

ربات تعاملی سلام

دانشجو:

علی کاشی پزها

استاد کارآموزی:

دکتر حمیدرضا تقی راد

محل کارآموزی:

صنایع پیشگام رباتیک ارس

آدرس:

دانشگاه صنعتی خواجه نصیرالدین طوسی، دانشکده برق

تابستان ۱۴۰۴

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

فهرست مطالب

عنوان	صفحه
فصل اول: معرفی صنایع پیشگام رباتیک ارس	5
1-1- مقدمه	5
2-1- پیشینه شرکت	5
فصل دوم: معرفی ربات تعاملی سلام	6
1-2- مقدمه	6
2-2- آشنایی با کلیات پروژه	6
3-2- ساختار فنی پروژه	7
1-3-2- سنسور ها	7
1-1-3-2- سونار (SRF05)	7
2-1-3-2- دوربین (Sony Playstation 3 Eye)	8
2-3-2- پردازنده ها	10
1-2-3-2- میکروکنترلر (Arduino Uno R3)	10
2-2-3-2- ریزپردازنده (Raspberry Pi 4)	11
3-3-2- نرم افزار ها	13
1-3-3-2- سیستم عامل (Raspberry Pi OS)	13
2-3-3-2- کتابخانه OpenCV	15
4-3-2- سخت افزار های جانبی	17
فصل سوم: پردازش تصویر و بینایی ماشین	18
1-3- مقدمه	18
2-3- پردازش تصویر	18
1-2-3- مفاهیم پایه در پردازش تصویر	18
2-2-3- تکنیک های پردازش تصویر	20
3-3- بینایی ماشین	21
1-3-3- الگوریتم های کلاسیک در بینایی ماشین	21

23	 یادگیری ماشین در بینایی ماشین
25	 فصل چهارم: نحوه اجرای پروژه با ماشین حالت متناهی و چندرسمانی
25	 1-4- مقدمه
25	 2-4- ماشین حالت متناهی ربات تعاملی سلام
26	 1-2-4- سیگنال ها و داده های ورودی به حالات
27	 2-2-4- توضیح عملیات انجام شده در هر حالت و نحوه انتقال بین حالات
33	 3-4- اجرای ماشین حالت متناهی با استفاده از چندرسمانی
39	 فصل پنجم: تشخیص حضور انسان با استفاده از آردوینو و سنسور سونار
39	 1-5- اتصال سونار به آردوینو و آردوینو به رزبری پای
39	 2-5- کدنویسی سنسور سونار در آردوینو
41	 3-5- کلاس Sonar در رزبری پای
43	 فصل ششم: تشخیص چهره و لباس
43	 1-6- تنظیم رنگ و پردازش فریم
51	 2-6- ساختار کلاس Camera
53	 1-2-6- کدنویسی تشخیص چهره
54	 2-2-6- کدنویسی تشخیص رنگ لباس
56	 3-2-6- اجرای مدل های تشخیص سن و جنسیت
59	 3-6- استفاده از Tensorflow C++ api برای آموزش مدل های دقیق تر
59	 1-3-6- دانلود دیتاست و نرمالسازی با تشخیص چهره
62	 2-3-6- آموزش مدل با کتابخانه Tensorflow در Python
68	 فصل هفتم: پخش فایل ویدئویی و صوتی
68	 1-7- فرآیند ها در سیستم عامل
69	 2-7- نحوه پخش فایل های ویدئویی و صوتی با مدیریت فرآیند ها
74	 3-7- دسترسی به فایل های ویدئویی و صوتی با داده های کلاس Camera
75	 منابع

چکیده

این گزارش به بررسی فعالیت‌های انجام‌شده در دوره کارآموزی در زمینه طراحی و پیاده‌سازی ربات تعاملی "سلام" پرداخته است. ربات مذکور با استفاده از تکنولوژی‌های پیشرفته در حوزه حسگرها و پردازش تصویر، به‌طور خودکار قادر است ویژگی‌های مختلف انسانی را شناسایی کرده و بر اساس آن‌ها اقدامات خاصی را انجام دهد. این ربات از حسگر سونار SRF02 برای تشخیص فاصله و مکان اشیاء اطراف خود استفاده می‌کند و از دوربین Sony PlayStation 3 Eye برای پردازش و تحلیل تصاویر انسان‌ها بهره می‌برد. پردازشگرهای این ربات شامل میکروکنترلر Arduino Uno برای مدیریت حسگر سونار و میکروپروسسور Raspberry Pi 4 برای پردازش تصاویر و انجام عملیات پیچیده‌تر است.

عملکرد ربات به‌گونه‌ای است که در ابتدا، حسگر سونار برای تشخیص حضور انسان‌ها در محیط فعال می‌شود و پس از تأیید حضور، دوربین به‌طور خودکار شروع به پردازش تصویر و شناسایی چهره فرد می‌کند. مدل‌های مختلف تشخیص چهره، شامل تشخیص سن، جنسیت، رنگ پوست و ویژگی‌های دیگر مانند رنگ مو، برای شناسایی و تجزیه و تحلیل چهره به کار گرفته می‌شوند. علاوه بر این، مدل LBF برای شناسایی و تحلیل نقاط کلیدی چهره استفاده شده است که کمک می‌کند ربات به‌دقت موقعیت‌هایی چون گوشه‌های چشم، دهان و نوک بینی را شبیه‌سازی کند.

در مواقعی که حسگر سونار نتواند داده دقیق‌تری ارائه دهد، سیستم از الگوریتم‌های تشخیص رنگ لباس و شناسایی ویژگی‌های بصری مانند رنگ لباس برای پخش صدا استفاده می‌کند. این قابلیت موجب می‌شود که ربات در مواقع خاص مانند شناسایی افراد در فاصله دور، همچنان قادر به تعامل و واکنش مناسب باشد. همچنین، ربات پس از شناسایی ویژگی‌ها، می‌تواند اقدامات خود را به‌طور خودکار تنظیم کرده و در صورت لزوم، ویدئو پخش یا صفحه نمایشگر را خاموش کند.

این پروژه، با ترکیب پردازش تصویر، تشخیص ویژگی‌های انسانی، و سیستم‌های خودکار، نمونه‌ای از تعاملات پیشرفته انسان-ربات را به نمایش می‌گذارد. هدف این گزارش، معرفی این ربات و فعالیت‌های انجام‌شده در طی دوره کارآموزی برای بهبود و توسعه تکنولوژی‌های مبتنی بر هوش مصنوعی و شبیه‌سازی است.

واژه‌های کلیدی:

Face detection, Face recognition, Sonar detection, Arduino uno, Raspberry Pi 4

فصل اول

معرفی صنایع پیشگام رباتیک ارس

1-1- مقدمه

شرکت پیشگامان رباتیک ارس به عنوان بازوی تجاری گروه رباتیک ارس دانشگاه خواجه نصیرالدین طوسی دارای بیش از 20 سال تجربه در حوزه ارائه راهکارهای پیشرفته و رباتیک در پاسخ به مسائل مختلف صنعتی می باشد. موفقیت امروز این شرکت عمدتاً به خاطر دانشجویان متعهد و اعضای فارغ التحصیل است که زمان تحقیق خود را در این موضوعات صرف کردند. علاوه بر این، همکاران ملی و بین المللی، امکانات پژوهشی و کارگاهی، و موضوعات تحقیقاتی پیشرفته از جمله دلایل اصلی موفقیت درخشان ارس در خاورمیانه هستند.

1-2- پیشینه شرکت

دوربین های عنکبوتی و کابلی: فیلم برداری به وسیله دوربین های عنکبوتی هم اکنون جزء پیشرفته ترین روش های فیلم برداری در محیط های باز و گسترده می باشد. این شیوه فیلم برداری با ایجاد قاب های تصویری زیبا جلوه ای دیگر به هنر شما می دهد.

تبلیغات BTL به وسیله ربات: تبلیغات BTL ارتباط مستقیم بین برندها و مشتریان را امکان پذیر می کند. ارتباط مستقیم به مشتریان کمک می کند تا محصول را به روشی ملموس تر درک کنند. این به برندها کمک می کند تا الگوهای خرید و رفتارهای مشتریان را درک کنند.

هوش مصنوعی و صنعت: بر اساس تحقیقات تیم های فروشی که از هوش مصنوعی استفاده می کنند توانسته اند تا ۵۰٪ سرنخ های فروش خود را افزایش دهند. هوش مصنوعی به فروشندگها کمک می کند که زمان کمتری را صرف ثبت لیدها، صلاحیت سنجی آنها، پیگیری و ایجاد رابطه با آنها کنند.

نرم افزار پیشبینی فروش فانوس- بر مبنای AI: امروزه پیش بینی دقیق به یکی از دغدغه های مهم موسسات تجاری تبدیل شده است همه کسب و کارها نیازمند پیشبینی میزان فروش، تقاضا و قیمت محصولات خود هستند تا بر اساس آن یک برنامه ریزی دقیق برای تولید، رقابت و ارائه خدمات داشته باشند.

فصل دوم

معرفی ربات تعاملی سلام

2-1- مقدمه

در این بخش، به طور مختصر اهداف پروژه، زمینه فعالیت ها و اهمیت شان توضیح داده خواهد شد. همچنین به معرفی ابزارها، تئوری ها و نرم افزارهایی که در طول دوره استفاده کرده ام می پردازم.

2-2- آشنایی با کلیات پروژه:

هدف اصلی پروژه **Salam Robot** ارائه تبلیغات هدفمند و بهبود تعاملات با توجه به خصوصیات چهره افراد بود تا این ربات با استفاده از سنسورها و الگوریتم های پیشرفته به تشخیص و تجزیه و تحلیل ویژگی های صورت افراد پرداخته و اطلاعات مرتبط را برای ارائه تبلیغات شخصی سازی شده استفاده کند. بدین ترتیب، تبلیغات باید متناسب با ویژگی های ظاهری فرد مانند سن و جنسیت نمایش داده شود. این روند شامل مراحل زیر است:

تشخیص چهره و ویژگی های صورت: ربات با استفاده از دوربین و سنسور سونار قادر به شناسایی چهره افراد و تحلیل ویژگی هایی مانند سن و جنسیت است. این اطلاعات برای شناسایی دقیق تری از مخاطب استفاده می شود. به عنوان مثال، سنسور سونار ابتدا حضور فرد را تشخیص می دهد و سپس دوربین وارد عمل می شود تا ویژگی های صورت مانند سن، جنسیت را تشخیص دهد.

تحلیل داده ها: با استفاده از مدل های هوش مصنوعی و یادگیری عمیق، ربات قادر به پیش بینی اطلاعاتی مانند سن (کودک، بزرگسال، سالمند) و جنسیت (مرد یا زن) فرد خواهد بود. این اطلاعات به صورت لحظه ای تحلیل می شود تا بهترین نوع تبلیغ برای هر فرد ارائه شود. این مرحله شامل استفاده از الگوریتم های پیچیده برای تجزیه و تحلیل ویژگی های چهره و تبدیل آن به داده های مفید برای ارائه تبلیغات است.

ارائه تبلیغات شخصی سازی شده: بر اساس ویژگی های تشخیص داده شده، ربات تبلیغاتی که به نوع فرد و نیازهای احتمالی اش مربوط است، پخش می کند. این تبلیغات می تواند شامل ویدئوها و صداهایی باشد که به طور ویژه برای هر فرد و موقعیت تنظیم شده است. به عنوان مثال، اگر فردی شبیه به یک کودک تشخیص داده شود، تبلیغات مرتبط با اسباب بازی ها یا محصولات کودکانه به او نمایش داده خواهد شد.

بهبود تجربه کاربری: هدف از این پروژه، بهبود تجربه کاربری از طریق ارائه تبلیغات هدفمند و متناسب با ویژگی‌های چهره فرد است. این امر می‌تواند در زمینه‌های مختلفی مانند تبلیغات تجاری، بازاریابی دیجیتال و حتی ارتباطات در محیط‌های عمومی مانند فروشگاه‌ها، نمایشگاه‌ها و مراکز خرید بسیار مؤثر باشد. با استفاده از این سیستم، تعاملات بین ربات و افراد به شکل شخصی‌تر و هدفمندتر خواهد بود، که به نوبه خود می‌تواند اثربخشی تبلیغات را افزایش دهد.

در ابتدا پروژه ربات تعاملی سلام محدود به تشخیص حضور افراد در محدوده سنسور سونار و ارائه تبلیغات تنها با توجه به مدل‌های تشخیص سن و جنسیت بود. اما در ادامه مسیر ایده‌های خودم را به پروژه اضافه کردم.

2-3- ساختار فنی پروژه:

2-3-1- سنسورها:

2-3-1-1- سونار (SRF05): برای تشخیص حضور افراد در نزدیکی ربات. استفاده شده است. سنسور التراسونیک SRF05 (شکل 2-3-1-1) یکی از پرکاربردترین حسگرها برای اندازه‌گیری فاصله است که با استفاده از امواج صوتی با فرکانس بالا (در حدود 40 کیلوهرتز) عمل می‌کند. این سنسور دارای دو مبدل فراصوت (Transducers) و دو پایه کنترلی اصلی (Trigger و Echo) است که در ادامه به تفکیک توضیح داده می‌شوند.

مبدل فرستنده (Transmitting Transducer): مبدل فرستنده وظیفه‌ی تولید و ارسال امواج فراصوتی را بر عهده دارد. با اعمال یک سیگنال الکتریکی از طرف مدار کنترلی، این مبدل آن را به ارتعاشات مکانیکی در محدوده‌ی فراصوت تبدیل کرده و موج تولید شده را به محیط ارسال می‌کند.

مبدل گیرنده (Receiving Transducer): مبدل گیرنده نقش دریافت‌کننده‌ی امواج بازتابی (Echo) را دارد. هنگامی که موج فراصوت پس از برخورد به مانع بازتاب می‌یابد، این مبدل آن را دریافت کرده و به سیگنال الکتریکی قابل پردازش تبدیل می‌نماید.

پایه Trig: پایه Trigger ورودی سنسور محسوب می‌شود و برای آغاز فرآیند اندازه‌گیری به کار می‌رود. برای فعال‌سازی این پایه، میکروکنترلر باید یک پالس منطقی با عرض حدود 10 میکروثانیه به آن اعمال کند. در پی این تحریک، مبدل فرستنده شروع به ارسال موج فراصوتی می‌کند.

پایه Echo: پایه Echo خروجی سنسور است که مدت زمان رفت و برگشت موج فراصوت را به صورت یک

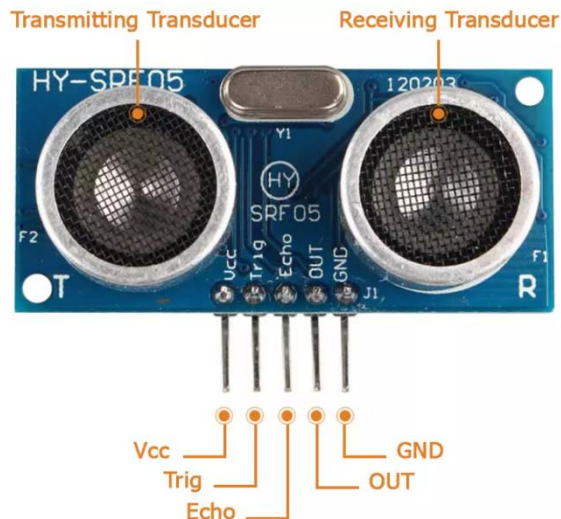
پالس منطقی نشان می‌دهد. به این صورت که پس از ارسال موج از طریق پایه Trigger، پایه Echo در وضعیت High قرار می‌گیرد و در حالت High باقی می‌ماند تا لحظه‌ای که موج بازتابی توسط مبدل گیرنده دریافت شود. مدت زمان این وضعیت High متناسب با زمان رفت و برگشت موج و در نتیجه متناسب با فاصله‌ی مانع از سنسور خواهد بود.

اصول اندازه‌گیری فاصله: زمان اندازه‌گیری شده توسط پایه Echo متناسب با مسافت طی شده توسط موج است. با توجه به این‌که موج مسیر رفت و برگشت را طی می‌کند، فاصله‌ی مانع از رابطه‌ی زیر محاسبه می‌شود:

$$\frac{\text{Time} \times \text{SpeedOfSound}}{2} = \text{Distance}$$

که در آن:

- **Time** مدت زمان پالس خروجی پایه Echo بر حسب میکروثانیه است.
- **Speed of Sound** سرعت صوت در هوا (تقریباً 343 متر بر ثانیه در دمای اتاق) است.



شکل 2-1-3-1: سنسور SRF05

2-1-3-2- دوربین (Sony Playstation 3 Eye): دوربین Sony Playstation 3 Eye (شکل 2-1-3-2) یک دوربین دیجیتال است که در ابتدا برای کنسول بازی Playstation 3 طراحی شده بود. این دوربین مجهز به یک سنسور CMOS با دقت 640×480 پیکسل است که قادر به ضبط ویدئو با نرخ

فریم 60 فریم بر ثانیه می‌باشد. این دوربین همچنین از ویژگی‌هایی نظیر قابلیت تشخیص حرکت و دستورات صوتی بهره‌مند است، که آن را به ابزاری کاربردی در پروژه‌های مختلف، به ویژه در زمینه‌های تحقیقاتی و علمی، تبدیل می‌کند.

یکی از دلایل اصلی استفاده از این دوربین در پروژه‌های دانشجویی، ارزان بودن و در دسترس بودن آن است. در حالی که بسیاری از دوربین‌های دیگر در بازار با قیمت‌های بالاتر و ویژگی‌های پیچیده‌تری عرضه می‌شوند، دوربین PlayStation 3 Eye با قابلیت‌های قابل قبول برای استفاده در پروژه‌های مختلف، به گزینه‌ای جذاب تبدیل شده است. این دوربین برای دانشجویانی که به دنبال ابزار ساده اما کاربردی برای شبیه‌سازی، شناسایی حرکت، و حتی شبیه‌سازی سیستم‌های واقعیت افزوده یا واقعیت مجازی هستند، بسیار مناسب است.

همچنین، این دوربین به راحتی می‌تواند با استفاده از نرم‌افزارهای مختلف مانند OpenCV و سایر ابزارهای پردازش تصویر و شبیه‌سازی حرکت به کار گرفته شود. این ویژگی‌ها باعث می‌شود تا از آن در پروژه‌های مرتبط با بینایی ماشین، شبیه‌سازی حرکت، تشخیص دستورات صوتی و به طور کلی پروژه‌های مربوط به هوش مصنوعی و رباتیک استفاده شود.

در مجموع، دوربین PlayStation 3 Eye به دلیل ویژگی‌های خاص، قیمت مناسب، و قابلیت‌های متنوع آن، به یک ابزار محبوب در بین دانشجویان و محققان تبدیل شده است و در بسیاری از پروژه‌های تحقیقاتی و دانشجویی مورد استفاده قرار می‌گیرد.



شکل 2-3-1: دوربین Sony Playstation 3 Eye

2-3-2- پردازنده ها:

2-3-2-1- میکروکنترلر (Arduino Uno R3): میکروکنترلر یک سیستم کامپیوتری کوچک و یکپارچه است که شامل پردازنده مرکزی (CPU)، حافظه (RAM و ROM)، و ورودی/خروجی ها (I/O) است. میکروکنترلرها معمولاً برای کنترل و نظارت بر دستگاه‌ها و سیستم‌های الکترونیکی طراحی شده‌اند و به عنوان مغز دستگاه‌های مختلف مانند ماشین‌های خانگی، لوازم الکترونیکی، ربات‌ها، سیستم‌های خودکار، و تجهیزات پزشکی عمل می‌کنند. میکروکنترلرها معمولاً در یک تراشه مجتمع قرار دارند که تمامی اجزای مورد نیاز برای عملکرد دستگاه را فراهم می‌آورد.

این تراشه‌ها می‌توانند ورودی‌ها را از سنسورها دریافت کرده و پردازش کنند، سپس بر اساس پردازش انجام شده، خروجی‌ها را به عملگرها یا دستگاه‌های دیگر ارسال کنند. میکروکنترلرها در بسیاری از پروژه‌های الکترونیکی نقش اساسی دارند. استفاده از میکروکنترلرها در سیستم‌های embedded (نهفته) به دلیل کارایی بالا، مصرف انرژی کم، هزینه پایین و اندازه کوچک آن‌ها بسیار محبوب است.

Arduino Uno R3 (شکل 2-3-2-1) یکی از محبوب‌ترین میکروکنترلرهای موجود در بازار است که به طور گسترده‌ای در پروژه‌های آموزشی، تحقیقاتی، و صنعتی استفاده می‌شود. این برد توسعه‌دهنده از میکروکنترلر ATmega328P استفاده می‌کند و به دلیل ویژگی‌های منحصر به فرد خود، به یکی از ابزارهای اصلی در ساخت سیستم‌های تعبیه‌شده تبدیل شده است. برد Arduino Uno R3 مجهز به 14 پین ورودی/خروجی دیجیتال، 6 پین ورودی آنالوگ، یک پورت USB برای برنامه‌ریزی، و امکانات ارتباطی ساده است که آن را برای استفاده در پروژه‌های متنوع بسیار مناسب می‌سازد.

یکی از ویژگی‌های برجسته Arduino Uno R3، دسترسی به محیط توسعه نرم‌افزاری ساده و کاربرپسند Arduino IDE است که به برنامه‌نویسان این امکان را می‌دهد تا برنامه‌های خود را به راحتی با استفاده از زبان برنامه‌نویسی ++C/C نوشتن و بر روی برد بارگذاری کنند. این زبان برنامه‌نویسی به صورت خاص برای استفاده در پروژه‌های ساده و پیچیده طراحی شده و به کاربران این امکان را می‌دهد تا به سرعت و به آسانی پروژه‌های مختلف را توسعه دهند.

برد Arduino Uno R3 به دلیل سادگی در استفاده، پشتیبانی گسترده از کتابخانه‌ها و پروژه‌های آنلاین، و توانایی سازگاری با انواع سنسورها و ماژول‌ها، به یکی از انتخاب‌های اصلی در میان دانشجویان و محققان برای طراحی و پیاده‌سازی پروژه‌های الکترونیکی و رباتیک تبدیل شده است. علاوه بر این، وجود پلتفرم‌های آموزشی و منابع گسترده آنلاین باعث شده که این میکروکنترلر به ابزاری ایده‌آل برای پروژه‌های آموزشی تبدیل شود.



شکل 2-3-2: برد Arduino Uno R3

2-2-3-2- ریزپردازنده (Raspberry Pi 4): ریزپردازنده‌ها واحدهای پردازشی مرکزی (CPU) هستند که برای انجام پردازش‌های پیچیده در سیستم‌های رایانه‌ای و الکترونیکی طراحی شده‌اند. این واحدهای پردازشی به‌طور گسترده‌ای در سیستم‌های پیچیده مانند کامپیوترها، سرورها، و سیستم‌های رایانه‌ای پیشرفته به‌کار می‌روند. برخلاف میکروکنترلرها، که معمولاً برای انجام وظایف خاص و ساده طراحی می‌شوند، ریزپردازنده‌ها برای انجام پردازش‌های عمومی و پیچیده‌تر در سیستم‌های رایانه‌ای و به‌ویژه برای اجرای سیستم‌عامل‌ها و نرم‌افزارهای پیچیده به‌کار می‌روند. به عبارت دیگر، ریزپردازنده‌ها به‌طور معمول قادر به پردازش داده‌ها و اجرای برنامه‌های پیچیده در سیستم‌های کامپیوتری هستند و نمی‌توانند به تنهایی به‌عنوان یک سیستم خودکفا عمل کنند. در واقع، ریزپردازنده‌ها معمولاً نیاز به اجزای جانبی دارند تا بتوانند عملکرد کامل خود را در یک سیستم رایانه‌ای یا الکترونیکی به انجام رسانند.

ریزپردازنده‌ها معمولاً تنها شامل واحد پردازش مرکزی (CPU) نمی‌شوند و برای انجام وظایف پردازشی به اجزای دیگری نظیر حافظه (RAM و ROM) و ورودی/خروجی‌های مختلف نیاز دارند. این واحدهای پردازشی در سیستم‌های رایانه‌ای و دستگاه‌های پیشرفته به‌منظور اجرای سیستم‌عامل‌ها، نرم‌افزارهای پیچیده، و فرآیندهای محاسباتی گوناگون به‌کار می‌روند. در نتیجه، این پردازنده‌ها عموماً برای انجام وظایف پردازشی عمومی و سیستم‌های پیچیده‌تر طراحی شده‌اند و به دلیل نیاز به اجزای اضافی برای تشکیل یک سیستم کامل، از نظر ساختاری با میکروکنترلرها متفاوت هستند.

تفاوت میان ریزپردازنده و میکروکنترلر

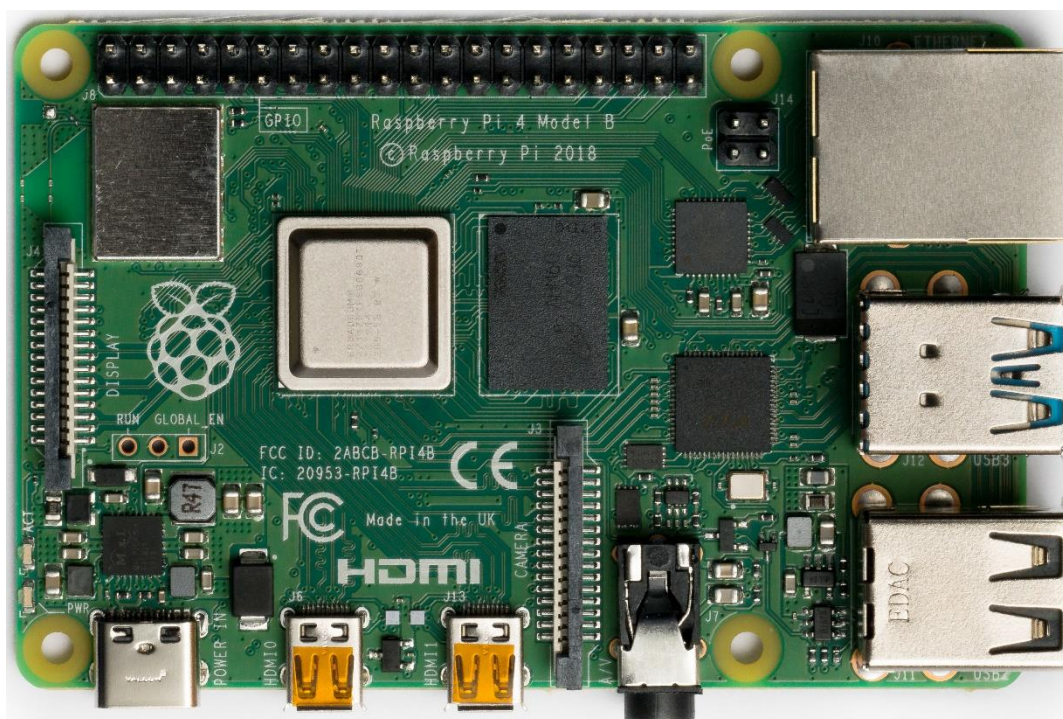
تفاوت اصلی میان ریزپردازنده‌ها و میکروکنترلرها در نوع کاربرد و پیچیدگی سیستم‌های مبتنی بر این دو است. ریزپردازنده‌ها معمولاً در سیستم‌هایی استفاده می‌شوند که نیاز به پردازش‌های پیچیده‌تر و اجرای نرم‌افزارهای سنگین دارند. این پردازنده‌ها برای پردازش داده‌ها در سیستم‌های رایانه‌ای، سرورها و سایر دستگاه‌های پیچیده طراحی شده‌اند و اغلب به همراه اجزای جانبی مانند حافظه‌های دسترسی تصادفی (RAM)، حافظه‌های فقط خواندنی (ROM) و اجزای ورودی/خروجی برای تشکیل یک سیستم کامل به کار می‌روند. در مقابل، میکروکنترلرها معمولاً در سیستم‌های نهفته (embedded systems) و پروژه‌های الکترونیکی ساده‌تر به کار می‌روند که نیاز به پردازش‌های پیچیده ندارند. میکروکنترلرها به‌طور معمول شامل تمام اجزای مورد نیاز برای عملکرد خود در یک تراشه واحد هستند و برای وظایف خاصی مانند کنترل دستگاه‌ها، حسگرها، و عملگرها طراحی شده‌اند.

به‌طور خلاصه، ریزپردازنده‌ها به دلیل توان پردازشی بالاتر و قابلیت اجرای سیستم‌عامل‌ها و نرم‌افزارهای پیچیده، معمولاً در سیستم‌های پیچیده‌تر و پردازش‌های عمومی مورد استفاده قرار می‌گیرند، در حالی که میکروکنترلرها به‌عنوان یک سیستم خودکفا، برای پروژه‌های ساده‌تر و کاربردهای خاص طراحی شده‌اند.

Raspberry Pi 4 (شکل 2-2-3-2) یک ریزپردازنده قدرتمند است که برای اهداف آموزشی، تحقیقاتی و صنعتی طراحی شده و از معماری ARM Cortex-A72 بهره می‌برد. این ریزپردازنده چهار هسته‌ای با فرکانس 1.5 گیگاهرتز، قابلیت پردازش داده‌های پیچیده و اجرای سیستم‌عامل‌های گوناگون مانند Raspberry Pi OS و Ubuntu را داراست. یکی از ویژگی‌های برجسته Raspberry Pi 4، توانایی آن در پشتیبانی از دو نمایشگر K4 از طریق پورت‌های HDMI 2.0 است که این ویژگی آن را برای پروژه‌های گرافیکی، پردازش تصویری و مدیا مناسب می‌سازد. علاوه بر این، این برد شامل 2، 4 یا 8 گیگابایت حافظه RAM است که بسته به نوع کاربرد، می‌تواند انتخاب مناسبی برای پروژه‌های مختلف باشد.

Raspberry Pi 4 از پورت‌های ورودی/خروجی متنوعی مانند پورت‌های USB 3.0، Ethernet، و پین‌های GPIO برای ارتباط با دستگاه‌های جانبی و سنسورها پشتیبانی می‌کند. این ویژگی‌ها امکان اتصال به طیف وسیعی از دستگاه‌ها و ماژول‌ها را فراهم می‌آورد که از آن در پروژه‌های مختلف از جمله رباتیک، اینترنت اشیا (IoT)، پردازش تصویر و ... استفاده می‌شود. همچنین، Raspberry Pi 4 قادر به اجرای سیستم‌عامل‌های مختلف است که امکان انجام پردازش‌های سنگین‌تر را فراهم می‌آورد.

از نظر کاربرد، Raspberry Pi 4 به دلیل پردازش قدرتمند و پشتیبانی از سخت‌افزارهای متعدد، برای پروژه‌های نیازمند پردازش‌های پیچیده، شبیه‌سازی محیط‌های تعاملی، و ارتباطات گسترده با دستگاه‌های جانبی ایده‌آل است. در مقایسه با میکروکنترلرها، که عمدتاً برای پروژه‌های خاص و کم‌حجم طراحی می‌شوند، Raspberry Pi 4 یک پلتفرم توسعه‌دهنده است که می‌تواند سیستم‌های پیچیده‌تری را اجرا کند و به‌عنوان یک ریزپردازنده، توانایی پردازش داده‌ها را در سطوح بالاتر ارائه دهد.



شکل 2-2-3-2: برد Raspberry Pi 4

2-3-3- نرم افزار ها:

2-3-3-1- سیستم عامل (Raspberry Pi OS): Raspberry Pi OS، که پیش‌تر با نام Raspbian شناخته می‌شد، یک سیستم‌عامل مبتنی بر لینوکس است که به‌طور خاص برای بردهای Raspberry Pi طراحی و بهینه‌سازی شده است. این سیستم‌عامل بر پایه Debian Linux ساخته شده و به‌عنوان پلتفرم اصلی برای بسیاری از پروژه‌های تحقیقاتی و آموزشی مورد استفاده قرار می‌گیرد. با توجه به ویژگی‌های منحصر به فرد سخت‌افزاری بردهای Raspberry Pi، سیستم‌عامل Raspberry Pi OS به‌طور خاص برای استفاده بهینه از پردازنده‌ها، حافظه و پورت‌های ورودی/خروجی طراحی شده است.

در حوزه پردازش تصویر، Raspberry Pi OS ابزارها و کتابخانه‌های نرم‌افزاری پیشرفته‌ای را برای تجزیه و تحلیل و پردازش تصاویر در اختیار کاربران قرار می‌دهد. کتابخانه‌های معروفی مانند OpenCV به‌طور

گسترده‌ای در این سیستم‌عامل پشتیبانی می‌شوند که امکان تحلیل داده‌های تصویری را به‌صورت بلادرنگ (real-time) فراهم می‌آورد. این ویژگی‌ها، همراه با قدرت پردازش بالای Raspberry Pi 4، باعث می‌شود که این سیستم‌عامل برای پیاده‌سازی پروژه‌های پردازش تصویر که نیاز به الگوریتم‌های پیچیده تحلیل تصویر دارند، بسیار مناسب باشد.

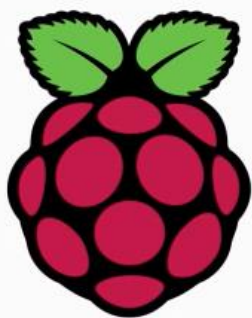
یکی از مزایای Raspberry Pi OS در پردازش تصویر، پشتیبانی از پردازش‌های گرافیکی و استفاده از پردازنده‌های گرافیکی (GPU) برای تسريع محاسبات است. این امر به‌ویژه در پروژه‌هایی که به تجزیه و تحلیل و شبیه‌سازی تصاویر در زمان واقعی نیاز دارند، حائز اهمیت است. در نتیجه، استفاده از Raspberry Pi OS به‌عنوان سیستم‌عامل اصلی در این پروژه‌ها، به‌ویژه در کاربردهایی همچون تشخیص الگو، شبیه‌سازی محیط‌های پیچیده، و سیستم‌های بینایی ماشین، توصیه می‌شود.

در پروژه‌هایی که به پردازش تصویر و تحلیل داده‌های سنسور نیاز دارند، ترکیب Raspberry Pi 4 به‌عنوان پلتفرم پردازشی با میکروکنترلر آردوینو به‌عنوان کنترل‌کننده دستگاه‌های ورودی، می‌تواند به‌عنوان یک راهکار قدرتمند و مقیاس‌پذیر عمل کند. در این پروژه‌ها، Arduino معمولاً برای کنترل سنسورها و جمع‌آوری داده‌ها از آن‌ها به کار می‌رود، در حالی که Raspberry Pi پردازش‌های پیچیده‌تر را انجام داده و وظیفه تحلیل و تجزیه و تحلیل داده‌های دریافتی را بر عهده دارد.

برای نمونه، در پروژه‌هایی که از سنسور سونار برای اندازه‌گیری فاصله استفاده می‌شود، آردوینو سیگنال‌های ارسالی از سنسور را دریافت کرده و بر اساس الگوریتم‌های مربوطه، فاصله را اندازه‌گیری می‌کند. این اطلاعات سپس از طریق سیگنال PWM به Raspberry Pi ارسال می‌شود. Raspberry Pi با استفاده از Raspberry Pi OS و ابزارهای پردازش تصویر، می‌تواند اطلاعات دریافتی از سنسور را تجزیه و تحلیل کرده و از نتایج برای انجام وظایف پیچیده‌تری استفاده کند.

در این ترکیب، Raspberry Pi 4 با استفاده از سیستم‌عامل Raspberry Pi OS و کتابخانه‌های پردازش تصویر همچون OpenCV قادر به انجام تحلیل‌های پیچیده مانند شناسایی اشیاء، پیگیری حرکت، و پردازش ویدئو به‌صورت بلادرنگ است. در عین حال، آردوینو به‌عنوان کنترل‌کننده سنسور، داده‌های مربوط به فاصله را با دقت بالا جمع‌آوری کرده و به سیستم پردازشی ارسال می‌کند.

این تقسیم وظایف بین دو سیستم Raspberry Pi و Arduino به آن‌ها این امکان را می‌دهد که به‌طور همزمان وظایف مختلفی را انجام دهند، بدون اینکه هر یک از آن‌ها تحت فشار پردازشی زیاد قرار گیرد. این طراحی بهینه، به‌ویژه در پروژه‌های پردازش تصویر و سیستم‌های بینایی ماشین که نیاز به پردازش داده‌ها در زمان واقعی دارند، بسیار کاربردی است.



Raspberry Pi OS

شکل 2-3-3-1 : لوگوی Raspberry Pi OS

2-3-3-2 کتابخانه OpenCV (Open Source Computer Vision Library):

کتابخانه نرم‌افزاری متن‌باز است که برای پردازش تصویر و بینایی ماشین طراحی شده است. این کتابخانه در ابتدا توسط Intel در سال 1999 توسعه یافت و از آن زمان به یکی از ابزارهای برجسته در پردازش تصویر دیجیتال، تحلیل و استخراج ویژگی‌ها، و شبیه‌سازی‌های رایانه‌ای تبدیل شده است. OpenCV به دلیل قابلیت‌های گسترده، عملکرد سریع و قابلیت استفاده در زبان‌های مختلف برنامه‌نویسی نظیر C++، Python، Java و Matlab، به‌طور گسترده‌ای در تحقیق و توسعه، به‌ویژه در حوزه‌های بینایی ماشین، رباتیک، شبیه‌سازی و پردازش ویدئو، استفاده می‌شود.

کتابخانه OpenCV شامل بیش از 2500 الگوریتم بهینه‌سازی شده است که بسیاری از آن‌ها قادر به انجام وظایف پیچیده‌ای مانند شناسایی ویژگی‌ها و اشیاء در تصاویر، پردازش تصاویر رنگی و سیاه‌وسفید، تحلیل ویدئو، و عملیات‌های هندسی پیچیده بر روی تصاویر هستند. این کتابخانه همچنین از الگوریتم‌های پیشرفته‌ای برای تکنیک‌های یادگیری ماشین و بینایی رایانه‌ای مانند پردازش تصویری، شناسایی و ردیابی اشیاء، تشخیص چهره، و عملکردهای پردازش ویدئو پشتیبانی می‌کند.

علاوه بر این، OpenCV امکانات گسترده‌ای برای پردازش تصاویر از فرمت‌های مختلف، همچنین قابلیت ارتباط با دوربین‌ها و دستگاه‌های ورودی تصویری، مانند وب‌کم‌ها، دوربین‌های دیجیتال و سنسورهای تصویر، فراهم می‌آورد. این قابلیت‌ها به OpenCV اجازه می‌دهند تا در پروژه‌های زمان‌واقع (real-time) پردازش تصویر و ویدئو به‌طور مؤثری استفاده شود.

کاربرد های OpenCV

OpenCV در بسیاری از زمینه‌های علمی و صنعتی کاربرد دارد و به‌ویژه در پروژه‌های بینایی ماشین و رباتیک نقش حیاتی ایفا می‌کند. برخی از مهم‌ترین کاربردهای OpenCV عبارتند از:

شناسایی و ردیابی اشیاء: OpenCV برای شناسایی و ردیابی اشیاء مختلف در تصاویر و ویدئوها به کار می‌رود. این الگوریتم‌ها می‌توانند برای شناسایی چهره‌ها، وسایل نقلیه، افراد، و حتی ردیابی حرکت اشیاء در ویدئوهای بلادرنگ استفاده شوند.

تشخیص ویژگی‌ها و تطابق تصاویر: از دیگر قابلیت‌های مهم OpenCV تشخیص ویژگی‌های هندسی مانند لبه‌ها، گوشه‌ها، و نقاط کانونی در تصاویر است. این ویژگی‌ها می‌توانند برای تطابق تصاویر مختلف، تحلیل تغییرات در تصاویر و یا فیلتر کردن داده‌ها در پردازش تصویر به کار روند.

پردازش ویدئو و فیلم: OpenCV همچنین برای تجزیه و تحلیل ویدئوهای زنده و ذخیره شده مورد استفاده قرار می‌گیرد. قابلیت‌هایی نظیر تشخیص حرکت، فیلتر کردن فریم‌ها، و پردازش تصویر به صورت بلادرنگ از ویژگی‌های کلیدی این کتابخانه هستند.

مزایای OpenCV

کارایی و سرعت بالا: یکی از ویژگی‌های برجسته OpenCV، سرعت پردازش بالای آن است. این کتابخانه به طور ویژه به گونه‌ای بهینه‌سازی شده که پردازش تصویر و ویدئو را در زمان‌های واقعی و با کمترین تأخیر ممکن انجام دهد.

کتابخانه متن‌باز: از آن‌جا که OpenCV متن‌باز است، این امکان را برای توسعه‌دهندگان فراهم می‌آورد که به راحتی به کدهای آن دسترسی داشته باشند و آن‌ها را برای نیازهای خاص خود سفارشی‌سازی کنند. این ویژگی باعث شده است که OpenCV در پروژه‌های تحقیقاتی به طور گسترده‌ای استفاده شود.

پشتیبانی از چندین زبان برنامه‌نویسی: OpenCV به طور کامل از زبان‌های برنامه‌نویسی مختلف مانند C++, Python, Java, Matlab، و Go پشتیبانی می‌کند که این امر به توسعه‌دهندگان این امکان را می‌دهد تا از آن در محیط‌های مختلف نرم‌افزاری استفاده کنند.

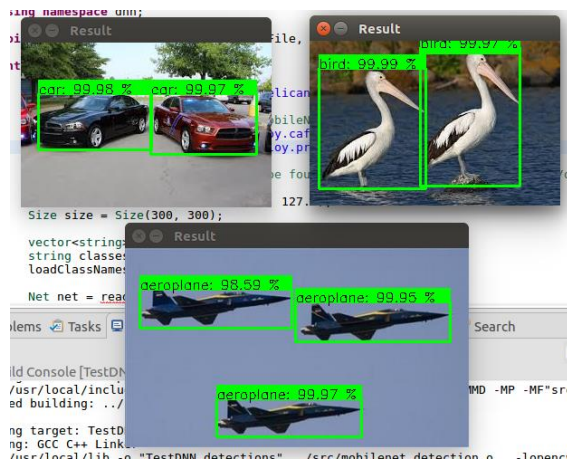
پشتیبانی از سخت‌افزارهای مختلف: OpenCV می‌تواند با طیف وسیعی از دستگاه‌ها، از جمله دوربین‌های دیجیتال، سنسورها، و پردازنده‌های گرافیکی (GPU) سازگار باشد، که این ویژگی باعث می‌شود تا برای انواع مختلف پروژه‌ها در صنایع گوناگون قابل استفاده باشد.

کتابخانه OpenCV شامل مجموعه‌ای گسترده از الگوریتم‌ها است که در زمینه‌های مختلف پردازش تصویر و بینایی ماشین به کار می‌روند. این الگوریتم‌ها شامل یادگیری ماشین، شبکه‌های عصبی کانولوشنی (CNN)، پردازش تصویر کلاسیک، و بینایی ماشین هستند که به طور مؤثر در تحلیل داده‌های تصویری و شبیه‌سازی‌های

پیچیده استفاده می‌شوند. استفاده از این الگوریتم‌ها و تکنیک‌ها در OpenCV به پژوهشگران و توسعه‌دهندگان این امکان را می‌دهد که پردازش‌های پیچیده‌ای را در حوزه‌های مختلف از جمله رباتیک، پردازش ویدئو، شبیه‌سازی محیط‌ها و بینایی ماشین انجام دهند.



شکل 2-3-3-2-ب : لوگوی OpenCV



شکل 2-3-3-2-الف : بینایی

کامپیوتر با استفاده از OpenCV

2-3-4- سخت افزار های جانبی:

2-3-4-1- فن و هیت سینک: برای خنک کردن واحد پردازش مرکزی Raspberry Pi.

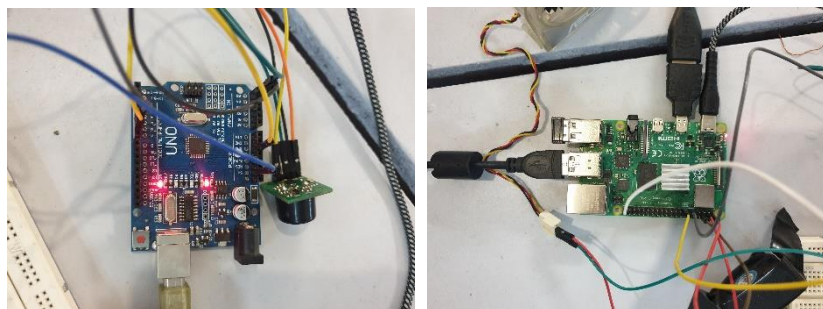
2-3-4-2- مانیتور: برای نمایش تبلیغ ها.

2-3-4-3- کابل HDMI و مبدل HDMI به micro-HDMI: برای اتصال رزبری به مانیتور.

2-3-4-3- برد مورد و سیم جامپر: برای اتصال سنسور سونار به آردوینو، آردوینو به رزبری و فن به رزبری.

2-3-4-4- آداپتور 15 وات و کابل USB-C: برای تغذیه رزبری. آردوینو و فن از طریق اتصال به رزبری

تغذیه می شوند.



شکل 2-3-4 : اتصالات رزبری پای (راست) و آردوینو (چپ)

فصل سوم

پردازش تصویر و بینایی ماشین

3-1- مقدمه

پردازش تصویر و بینایی ماشین از حوزه‌های مهم و پرکاربرد در علوم کامپیوتر و مهندسی برق هستند که به‌طور خاص به تجزیه و تحلیل تصاویر دیجیتال و استخراج اطلاعات مفید از آن‌ها می‌پردازند. پردازش تصویر و بینایی ماشین در طی دهه‌های اخیر پیشرفت‌های چشمگیری داشته‌اند و در کاربردهای متعددی از جمله رباتیک، سیستم‌های نظارت تصویری، پزشکی، شبیه‌سازی محیط‌های واقعی، خودروسازی و حتی در تعاملات انسان-کامپیوتر نقش حیاتی ایفا می‌کنند.

3-2- پردازش تصویر

هدف اصلی پردازش تصویر استخراج اطلاعات معنی‌دار از تصاویر و تبدیل آن‌ها به داده‌های قابل استفاده برای سیستم‌های رایانه‌ای است. این حوزه با توجه به پیشرفت‌های چشمگیر در فناوری‌های سخت‌افزاری و نرم‌افزاری در دهه‌های اخیر، در زمینه‌های مختلف علمی و صنعتی کاربردهای وسیعی پیدا کرده است.

3-2-1- مفاهیم پایه در پردازش تصویر

پردازش تصویر به‌طور کلی به مجموعه‌ای از عملیات‌های ریاضیاتی اطلاق می‌شود که به‌منظور بهبود، تحلیل، و استخراج اطلاعات از تصاویر دیجیتال انجام می‌شود. یک تصویر دیجیتال در واقع یک ماتریس از پیکسل‌ها است که هر پیکسل شامل مقدار مشخصی از رنگ یا روشنایی است. پردازش تصویر، به این ماتریس‌ها عملیات‌هایی مانند فیلتر کردن، تبدیل‌های هندسی، و تجزیه و تحلیل آماری را اعمال می‌کند تا ویژگی‌های خاص یا اطلاعات مفید از آن‌ها استخراج شود.

ویژگی‌های تصویر دیجیتال

- رزولوشن تصویر: تعداد پیکسل‌ها در هر بعد تصویر که وضوح تصویر را مشخص می‌کند.

- Bit depth**
The number of colours and tones that 1 pixel can record

4bit
16 levels x3

6bit
64 levels x3

8bit
256 levels x3

24-bit "true color" displays
Each screen pixel is represented by three groups of eight pixels, for a total of 24 bits.

Blue Green Red

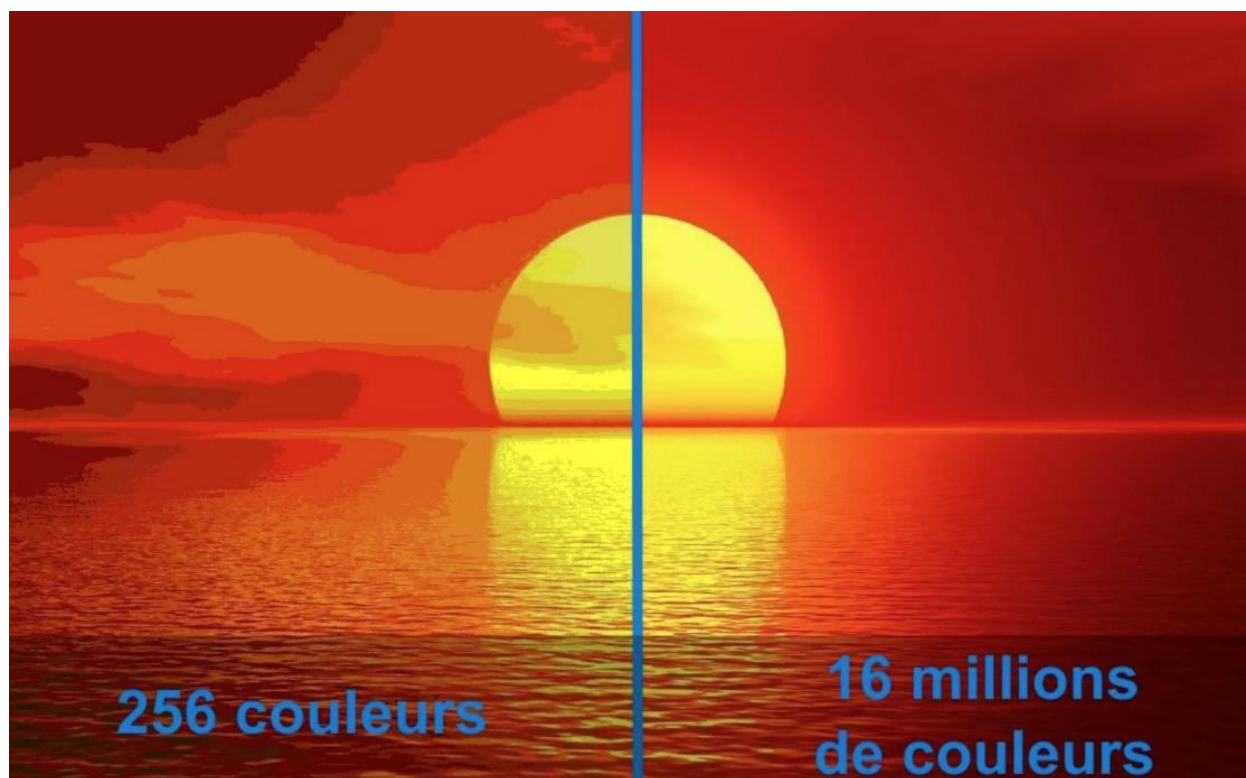
0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

Pixels on the computer screen

Picker
R: 255
G: 255
B: 0

Photoshop color picker shows the R, G, B components that make "yellow."

شکل 3-2-1 الف : عمق رنگ در کانال های RGB



19

3-2-2- تکنیک های پردازش تصویر

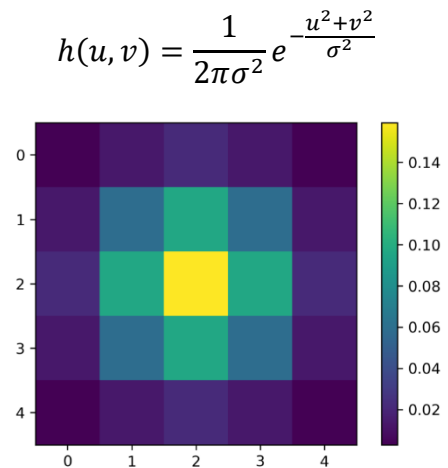
پردازش تصویر شامل مجموعه‌ای از الگوریتم‌ها است که به منظور انجام وظایف مختلفی مانند بهبود کیفیت تصویر، تحلیل ویژگی‌ها، و شناسایی الگوهای مختلف طراحی شده‌اند. مهم‌ترین آن‌ها عبارتند از:

فیلتر کردن تصویر (Image Filtering)

فیلتر کردن تصویر به منظور بهبود یا تغییر ویژگی‌های تصویر استفاده می‌شود. این فرآیند شامل اعمال یک فیلتر ریاضیاتی به هر پیکسل تصویر است که ممکن است منجر به کاهش نویز، تقویت لبه‌ها یا اعمال تغییرات دیگر در تصویر شود.

- **فیلتر گاوسی (Gaussian Filter):** برای کاهش نویز و هموارسازی تصویر به کار می‌رود. این فیلتر با استفاده از یک هسته گاوسی به هر پیکسل وزن‌دهی می‌کند تا ویژگی‌های تصویر صاف‌تر شوند.
- **فیلتر میانه (Median Filter):** برای حذف نویزهای "Salt-and-Pepper" از تصاویر استفاده می‌شود. در این روش، به جای استفاده از میانگین، میانگین میانه از مقادیر همسایگی انتخاب می‌شود.

Smoothing with a Gaussian



شکل 3-2-2- الف : فیلتر گاوسی

تحلیل و تغییر هندسی تصویر (Geometric Transformation)

تغییرات هندسی تصویر به منظور تغییر موقعیت یا شکل تصویر انجام می‌شود. این تغییرات شامل چرخش، تغییر مقیاس، کشش، انتقال و موارد مشابه است. این تکنیک‌ها برای اصلاح یا تنظیم موقعیت تصاویر در فرآیندهای پردازش و تجزیه و تحلیل استفاده می‌شوند.

- **تغییر مقیاس (Resizing):** تغییر ابعاد تصویر بدون تغییر محتوای آن.
- **چرخش (Rotation):** چرخش تصویر به یک زاویه مشخص.
- **انتقال (Translation):** جابجایی تصویر در جهات مختلف.

3-3- بینایی ماشین

هدف این حوزه شبیه‌سازی و مدل‌سازی فرآیندهای بینایی انسانی برای استخراج اطلاعات معنی‌دار از داده‌های تصویری است. بینایی کامپیوتر کاربردهای وسیعی در زمینه‌های مختلفی چون رباتیک، خودروسازی، پزشکی، نظارت تصویری، و شبیه‌سازی محیط‌های واقعی دارد.

3-3-1- الگوریتم‌های کلاسیک در بینایی ماشین

قبل از ظهور روش‌های یادگیری عمیق، الگوریتم‌های پردازش تصویر در بینایی ماشین مورد استفاده قرار می‌گرفتند که عمدتاً به عملیات‌های ریاضیاتی و هندسی محدود بودند. برخی از این الگوریتم‌ها عبارتند از:

تشخیص ویژگی‌ها (Feature Detection)

تشخیص ویژگی‌ها به فرآیند شناسایی نقاط مهم در یک تصویر اطلاق می‌شود که می‌توانند برای تجزیه و تحلیل بیشتر مورد استفاده قرار گیرند. ویژگی‌هایی مانند گوشه‌ها، لبه‌ها و نقاط کانونی می‌توانند اطلاعات کلیدی در مورد ساختار تصویر ارائه دهند.

- **تشخیص لبه (Edge Detection):** یکی از مهم‌ترین تکنیک‌ها در پردازش تصویر است. هدف از این تکنیک شناسایی مرزها و لبه‌های اشیاء در تصویر است. الگوریتم‌های مختلفی مانند **Canny Edge Detector** برای این منظور استفاده می‌شوند.
- **تشخیص گوشه‌ها (Corner Detection):** الگوریتم‌های **Harris Corner Detector** و **Shi-Tomasi** برای شناسایی نقاط گوشه‌ای و ویژگی‌های محلی استفاده می‌شوند. این ویژگی‌ها معمولاً در ردیابی اشیاء و تطابق تصاویر مفید هستند.

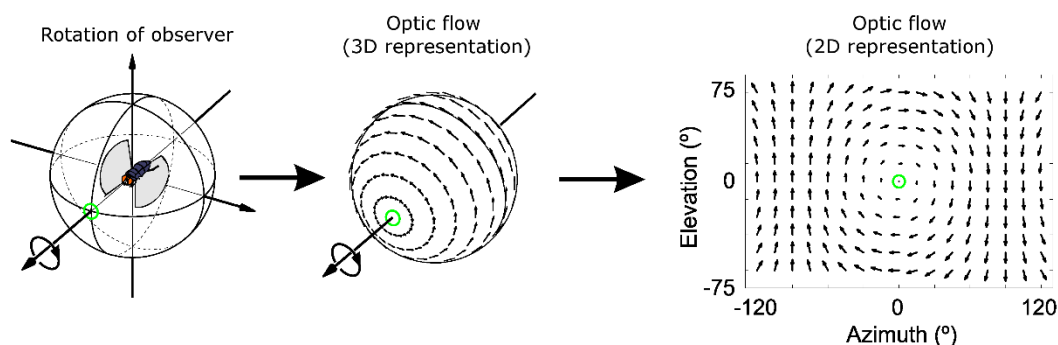


شکل 3-3-1 الف : تشخیص لبه

شبیه‌سازی و ردیابی حرکت (Motion Detection and Tracking)

ردیابی حرکت و شبیه‌سازی تغییرات در تصاویر و ویدئوها از دیگر بخش‌های مهم پردازش تصویر است. این فرآیند برای شناسایی و ردیابی اشیاء در ویدئوهای متحرک یا در زمان واقعی استفاده می‌شود.

- شبیه‌سازی حرکت (Motion Estimation): این فرآیند به شبیه‌سازی حرکت اشیاء در ویدئوها کمک می‌کند.
- ردیابی اشیاء (Object Tracking): استفاده از الگوریتم‌هایی مانند Kalman Filters و Optical Flow برای ردیابی موقعیت و حرکت اشیاء در ویدئوها.



شکل 3-3-1 ب : الگوریتم Optical Flow

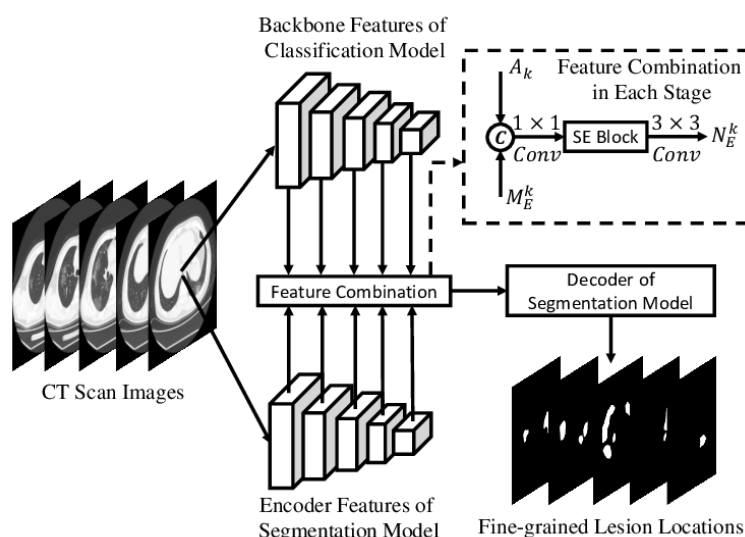
3-2- یادگیری ماشین در بینایی ماشین

یادگیری ماشین (Machine Learning) به مجموعه‌ای از الگوریتم‌ها و مدل‌های ریاضیاتی اطلاق می‌شود که به کامپیوترها این امکان را می‌دهند که از داده‌ها یاد بگیرند و بدون نیاز به برنامه‌نویسی مستقیم، پیش‌بینی‌هایی انجام دهند. از یادگیری ماشین برای شناسایی الگوها و ویژگی‌های پیچیده‌تر در تصاویر استفاده می‌شود.

تجزیه و تحلیل و طبقه‌بندی (Segmentation and Classification)

تجزیه و تحلیل تصویر به تقسیم‌بندی تصویر به بخش‌های مختلف اطلاق می‌شود که هر بخش نشان‌دهنده یک ویژگی خاص یا یک شیء است. این بخش‌ها سپس می‌توانند برای تحلیل یا طبقه‌بندی بیشتر استفاده شوند.

- تجزیه و تحلیل مبتنی بر مرز (Boundary-based Segmentation): تقسیم تصویر به نواحی مختلف بر اساس لبه‌ها و مرزهای موجود.
- تقسیم‌بندی مبتنی بر منطقه (Region-based Segmentation): تقسیم‌بندی تصویر بر اساس ویژگی‌های مشابه در نواحی مختلف.



شکل 3-2-الف: طبقه‌بندی تصویر

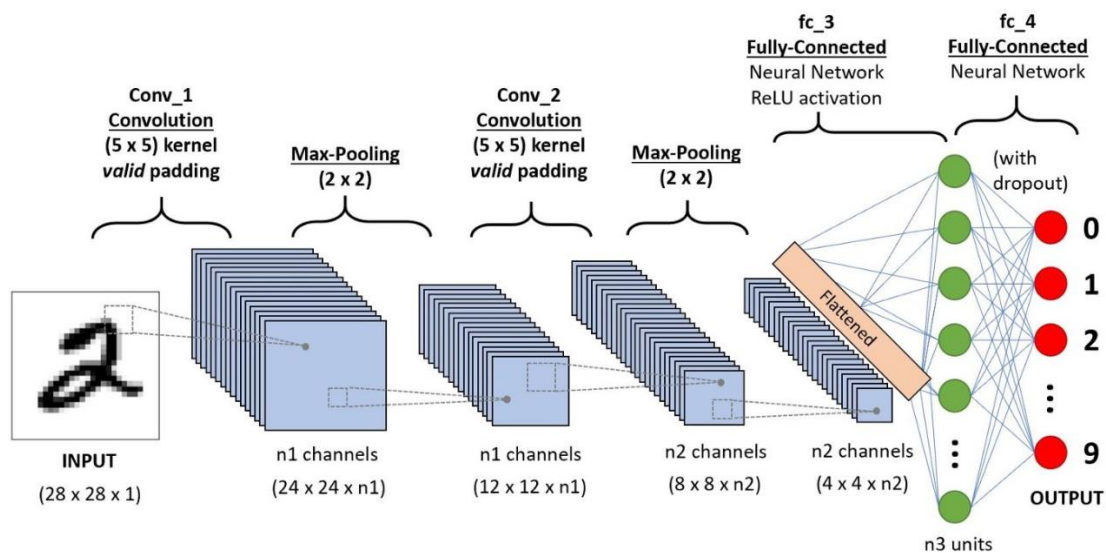
طبقه‌بندی تصویر به استفاده از الگوریتم‌های یادگیری ماشین یا یادگیری عمیق برای شناسایی و طبقه‌بندی اشیاء یا ویژگی‌ها در تصویر اطلاق می‌شود. این الگوریتم‌ها معمولاً از ویژگی‌های استخراج‌شده از تصویر برای تعیین طبقه یا برچسب استفاده می‌کنند.

شبکه‌های عصبی پیچشی (Convolutional Neural Networks - CNNs)

شبکه‌های عصبی پیچشی (CNNs) یکی از موفق‌ترین مدل‌های یادگیری عمیق در زمینه بینایی ماشین هستند. این مدل‌ها به‌ویژه برای شناسایی ویژگی‌ها و تحلیل تصاویر پیچیده طراحی شده‌اند. CNNها با استفاده از لایه‌های کانولوشن، لایه‌های pooling، و لایه‌های fully connected قادر به استخراج ویژگی‌های سطح پایین (مانند لبه‌ها و گوشه‌ها) و ویژگی‌های سطح بالا (مانند اشیاء و چهره‌ها) از تصاویر هستند.

- **لایه کانولوشنی (Convolutional Layer):** این لایه‌ها ویژگی‌های محلی مانند لبه‌ها و بافت‌ها را از تصویر استخراج می‌کنند.
- **لایه Pooling:** این لایه‌ها برای کاهش ابعاد و انتخاب ویژگی‌های برجسته از تصویر استفاده می‌شوند.
- **لایه Fully Connected:** این لایه‌ها برای طبقه‌بندی و پیش‌بینی نهایی مورد استفاده قرار می‌گیرند.

CNNها به‌ویژه در وظایفی همچون شناسایی اشیاء، طبقه‌بندی تصاویر، تشخیص چهره و تجزیه و تحلیل ویدئوهای متحرک از دقت بسیار بالایی برخوردارند.



شکل 3-3-2-ب: شبکه عصبی پیچشی

فصل چهارم

نحوه اجرای پروژه با ماشین حالت متناهی و چندریسمانی

4-1- مقدمه

در فصل دوم در مورد ساختار فنی پروژه اعم از سنسور ها، پردازنده ها و نرم افزار ها صحبت شد. در این فصل، به نحوه اجرای پروژه می پردازیم. ابتدا چگونگی اتصال بخش های اصلی آن به یکدیگر را بررسی و مرور کنیم:

- دوربین Sony Playstation 3 Eye به Raspberry Pi 4 متصل شده است.
- سنسور التراسونیک SRF05 به میکروکنترلر Arduino Uno R3 متصل شده است.
- میکروکنترلر آردوینو جهت ارسال سیگنال های pwm مربوط به سونار به رزبری پای متصل شده است.
- Raspberry Pi OS به عنوان سیستم عامل رسمی رزبری پای بر روی آن نصب شده و کتابخانه OpenCV جهت انجام پردازش های مربوط به تشخیص چهره و مدل های یادگیری ماشین برای تشخیص ویژگی های ظاهری نظیر سن و جنسیت با دریافت تصاویر از دوربین بر روی سیستم عامل نصب شده است.

4-2- ماشین حالت متناهی ربات تعاملی سلام

ماشین حالت متناهی (Finite State Machine یا FSM) یک مدل ریاضی برای تحلیل سیستم هایی است که در آن ها تعداد محدودی حالت وجود دارد و انتقال بین این حالات بر اساس ورودی ها و قوانین مشخص صورت می گیرد. این مدل در علوم کامپیوتر، مهندسی نرم افزار و سیستم های کنترلی به طور گسترده ای استفاده می شود.

FSM معمولاً از سه بخش اصلی تشکیل می شود:

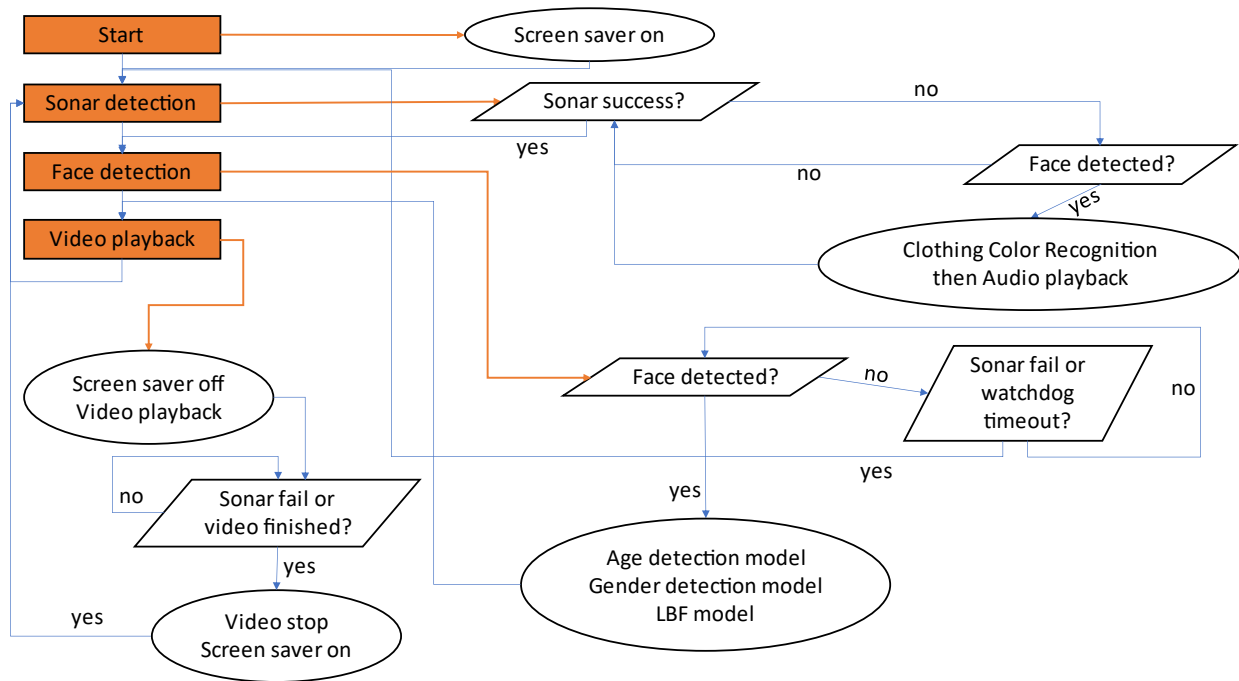
- **حالات (States):** مجموعه ای از حالات مختلف که سیستم می تواند در آن ها قرار گیرد.
- **ورودی ها (Inputs):** سیگنال ها یا داده هایی که بر اساس آن ها انتقال بین حالات انجام می شود.
- **انتقالات (Transitions):** قوانین یا روابطی که تعیین می کنند در پاسخ به ورودی ها، سیستم از یک حالت به حالت دیگر منتقل شود.

به طور کلی سه حالت اصلی را می توان برای ربات تعاملی سلام متصور شد:

- **حالت Sonar detection:** تشخیص حضور انسان یا شیء در نزدیکی ربات.
- **حالت Face detection:** پس از تشخیص حضور انسان، تشخیص چهره جهت تایید حضور انسان در نزدیکی ربات و سپس تشخیص ویژگی های ظاهری مانند سن و جنسیت.
- **حالت Video playback:** پس از تایید حضور انسان و تشخیص ویژگی های ظاهری او، نمایش تبلیغات منطبق با ویژگی های ظاهری تشخیص داده شده.

این حالات به صورت ترتیبی در ربات تعاملی سلام تعریف شده اند. یعنی اینکه فقط از Sonar detection به Face detection و فقط از Face detection به Video playback می توان انتقال یافت.

نحوه انتقال بین این حالات در شکل 4-2 به طور دقیق مشخص شده است.



شکل 4-2: فلوچارت ماشین حالت متناهی طراحی شده

4-2-1- سیگنال ها و داده های ورودی به حالات

دو سیگنال pwm شامل sonar.success و sonar.fail از طرف میکروکنترلر به رزبری پای ارسال می شوند. این سیگنال ها مطابق شکل 4-2 در تمام حالات استفاده و شرط آن ها بررسی می شود. سیگنال موفقیت زمانی ارسال می شود که برای 10 سیکل متوالی (هر سیکل 100 میلی ثانیه) فاصله محاسبه شده از طریق سنسور سونار کمتر از فاصله threshold باشد. سیگنال شکست نیز زمانی ارسال می شود که برای 10 سیکل متوالی

فاصله محاسبه شده بیشتر از threshold باشد. این یعنی زمانی که دستمان را مقابل سنسور به صورت عمودی جا به جا کنیم، هیچکدام از دو سیگنال high نمی شوند زیرا برای 10 سیکل متوالی شرط مطلوب را ندارند. متغیر camera.success نیز در حالت های Sonar detection و Face detection مورد استفاده قرار گرفته است. زمانی که مقدار true برگرداند یعنی چهره تشخیص داده شده و زمانی که false باشد یعنی چهره ای تشخیص داده نشده.

متغیر های watchdogTimer و watchdogTimeout نیز در Face detection استفاده می شوند. watchdogTimer زمان سپری شده در حالت را به ثانیه نگهداری می کند و با ورود به آن reset می شود. watchdogTimeout حداکثر مدت زمان نیست که در هر حالت گذرانده می شود. اگر watchdogTimer از watchdogTimeout بیشتر شود، به حالت Sonar detection باز می گردیم.

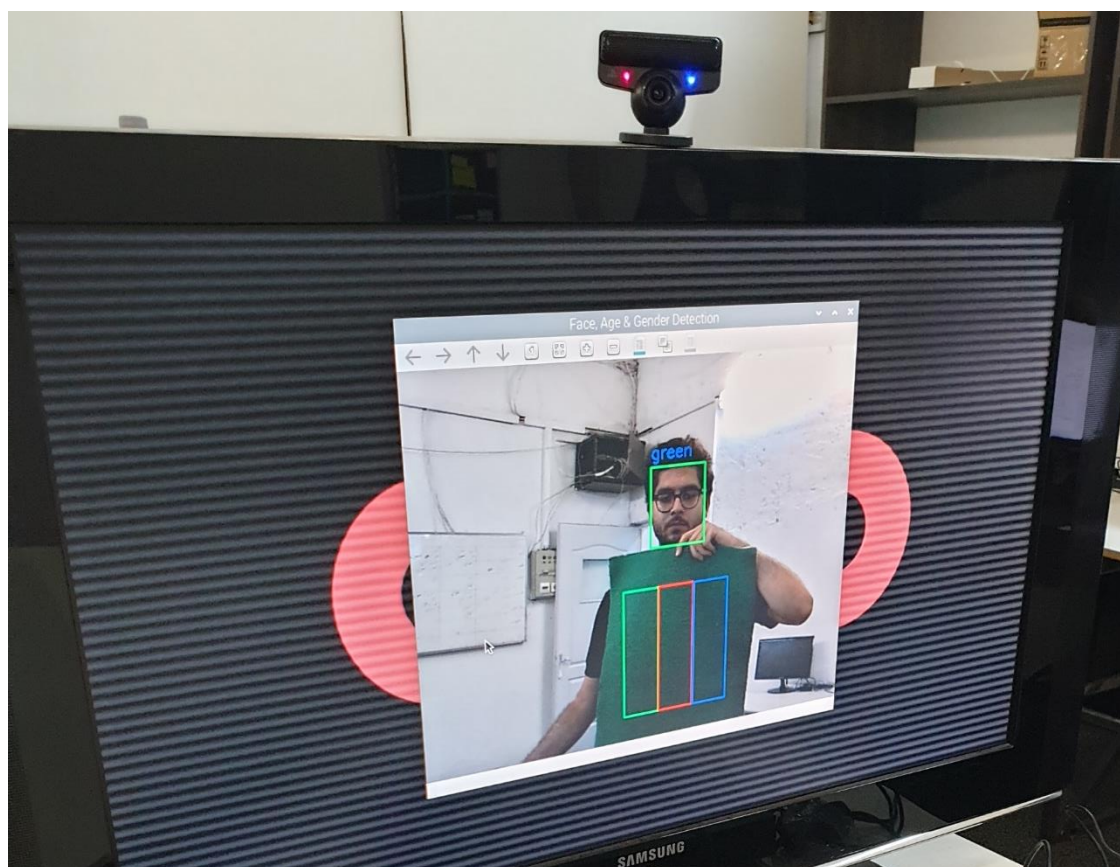
4-2-2- توضیح عملیات انجام شده در هر حالت و نحوه انتقال بین حالات

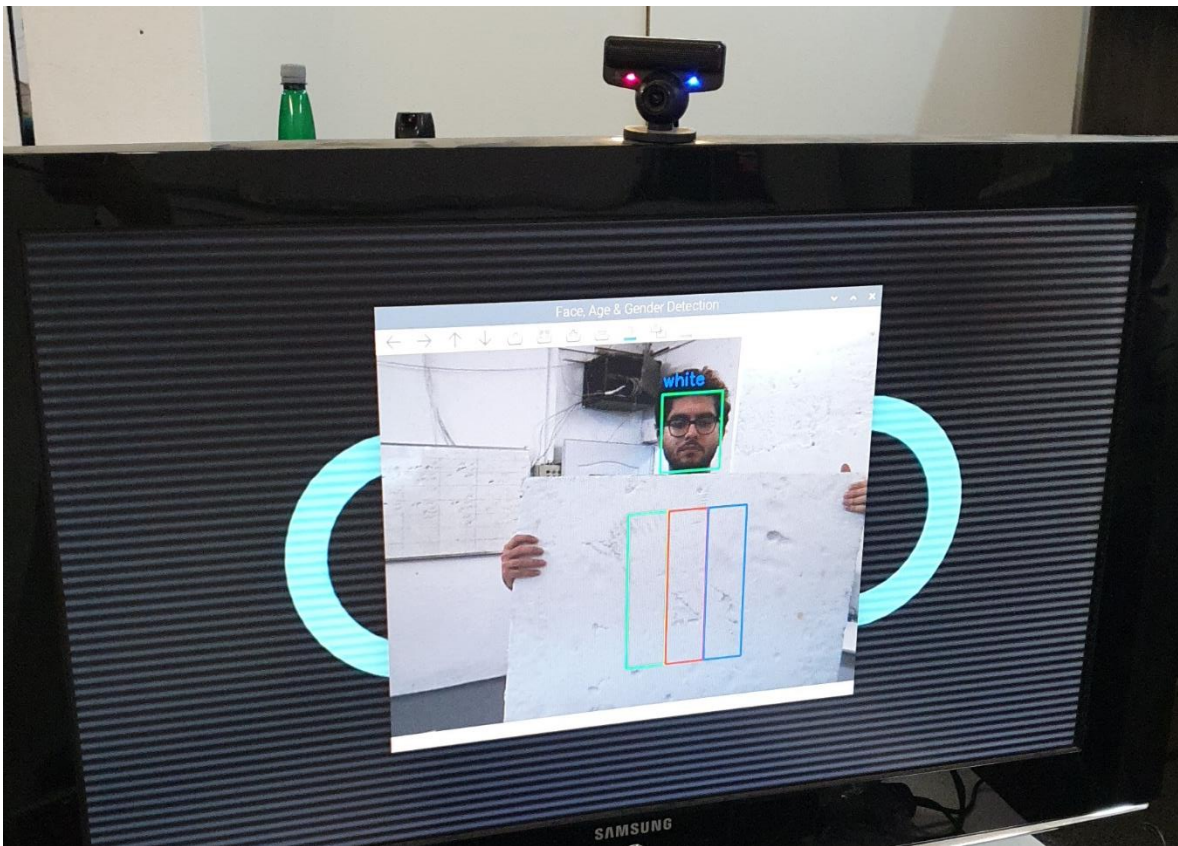
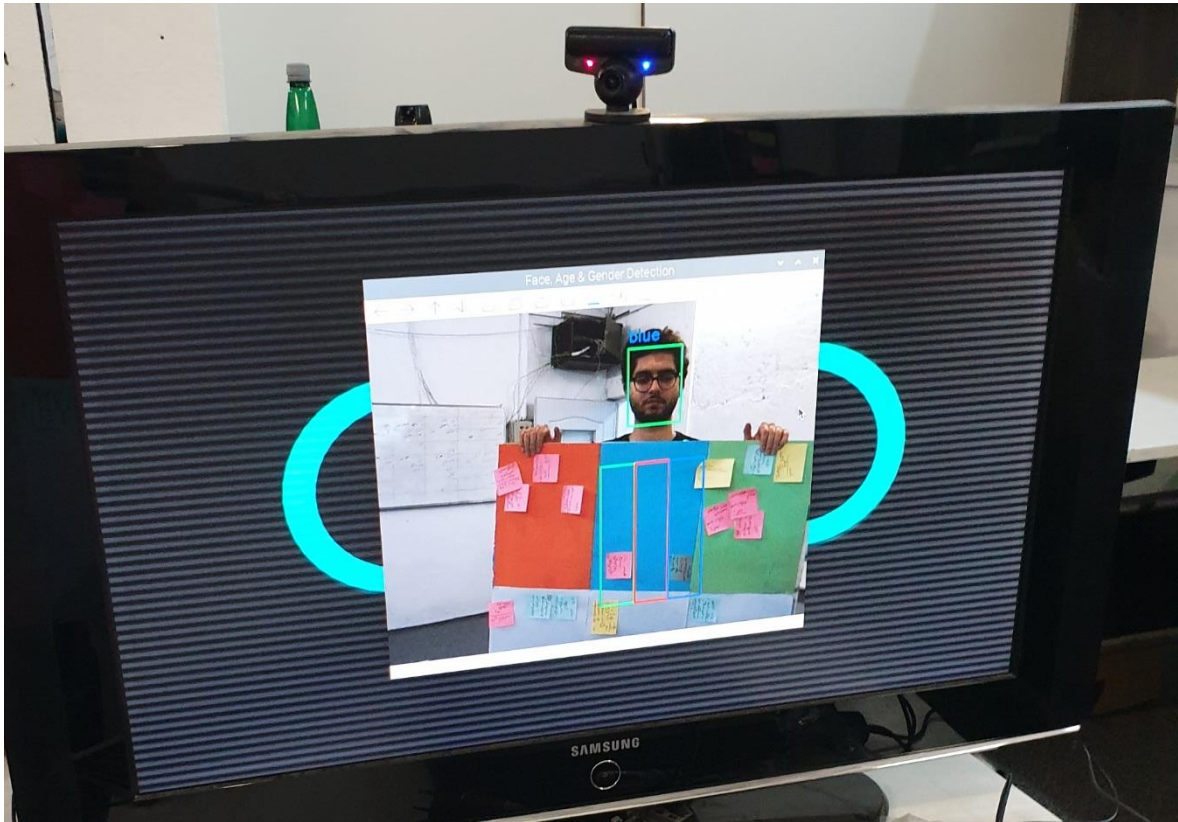
با توجه به شکل 4-2، در آغاز برنامه یک GIF مناسب به عنوان screen saver و جهت جلب توجه مشتریان هنگامی که تبلیغاتی نمایش داده نمی شوند بر روی صفحه مانیتور بالا می آید.

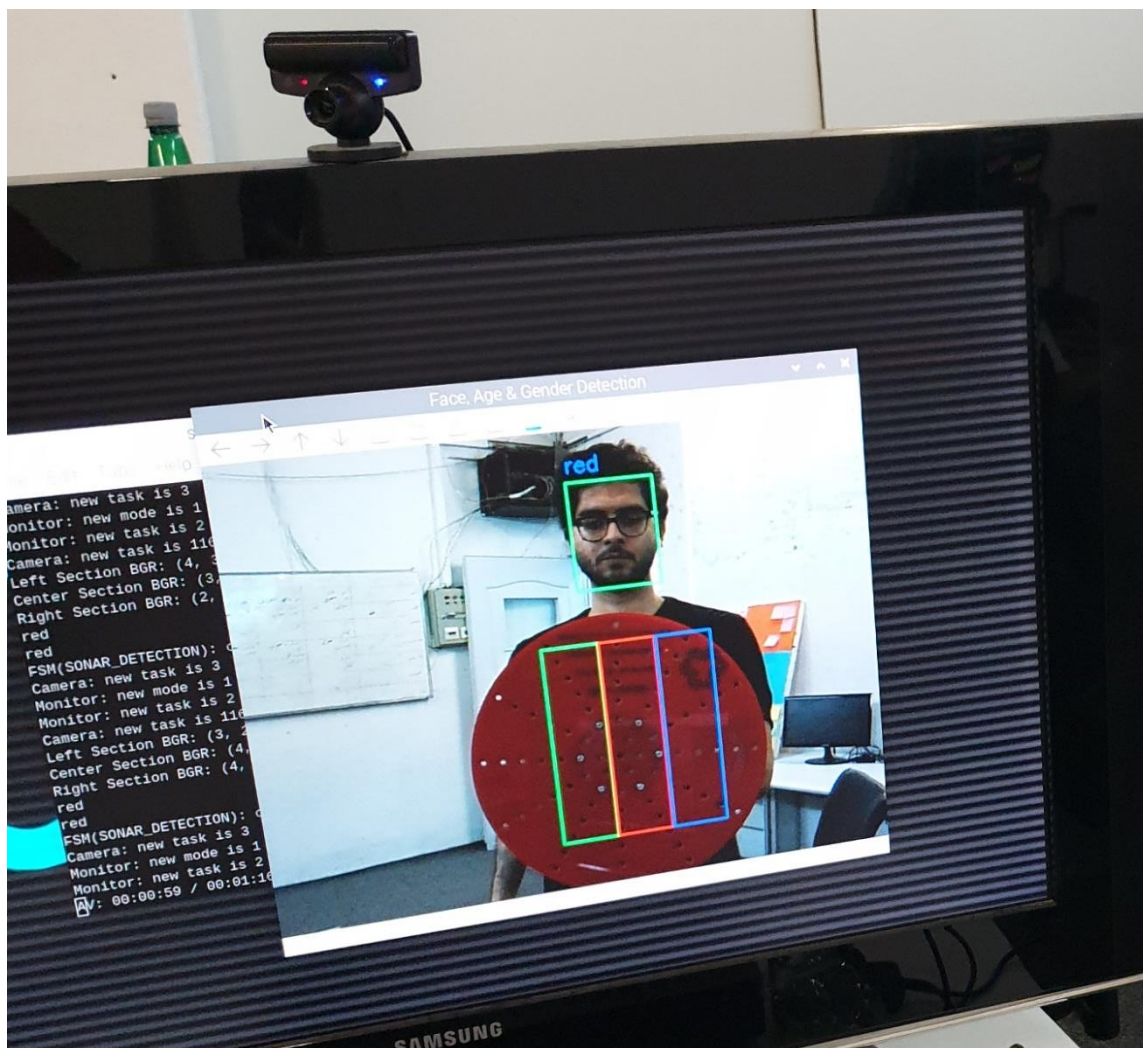
Sonar detection state

در حالت Sonar detection، رزبری پای منتظر دریافت سیگنال موفقیت سونار از میکروکنترلر آردوینو است. در صورت دریافت سیگنال، وارد حالت Face detection می شود. همزمان با دریافت سیگنال از میکروکنترلر، از طریق تصاویر دریافت شده از دوربین پردازش های تشخیص چهره را انجام می دهد تا در صورتی که سیگنال موفقیت را از میکروکنترلر دریافت نکرد، با تشخیص حضور افراد در فاصله دورتر توجه مشتریان را جلب کند. در این مرحله، در صورت تشخیص چهره، رنگ لباس فرد تشخیص داده شده را پردازش می کند تا با تشخیص رنگ لباس او فایل صوتی مخصوصی را پخش کند. برای نمونه، اگر فردی با لباس آبی در فاصله دور از ربات حضور دارد و سنسور سونار سیگنال موفقیت را صادر نمی کند، اگر الگوریتم های تشخیص چهره موفق به تشخیص چهره آن فرد شوند، فایل صوتی با محتوا هایی نظیر "تو که لباس آبی پوشیده ای، میخواهم یک تبلیغ بهت نشان بدهم."، "چقدر لباس آبی بهت میاد!"، "لباس آبی، میتوانم وقتت را بگیرم؟" و ... پخش خواهد شد و برای هر رنگ فایل صوتی مربوط آن در حافظه ریزپردازنده ذخیره شده است. اگر هم چهره ای تشخیص داده نشد یا رنگ لباس فرد قابل تشخیص نبود، بدون کار اضافه منتظر دریافت سیگنال موفقیت از میکروکنترلر جهت رفتن به حالت بعدی خواهد ماند. بین هر فرآیند تشخیص رنگ لباس و پخش فایل صوتی

چند ثانیه فاصله برقرار شده است تا از ایجاد سروصدا توسط ربات در محیط های شلوغ جلوگیری شود. (ایده پخش فایل صوتی در ابتدای پروژه وجود نداشت و خودم آن را به ربات تعاملی سلام اضافه کردم). عکس های زیر رنگ لباس های تشخیص داده شده برای پخش صوت حین اجرا را نمایش می دهند:







همانطور که در عکس ها قابل مشاهده است، در حالت Sonar detection رنگ های مشکی، سبز، سفید، آبی و قرمز با انتخاب roi (region of interest) مناسب برای لباس تشخیص داده شده است.

Face detection state

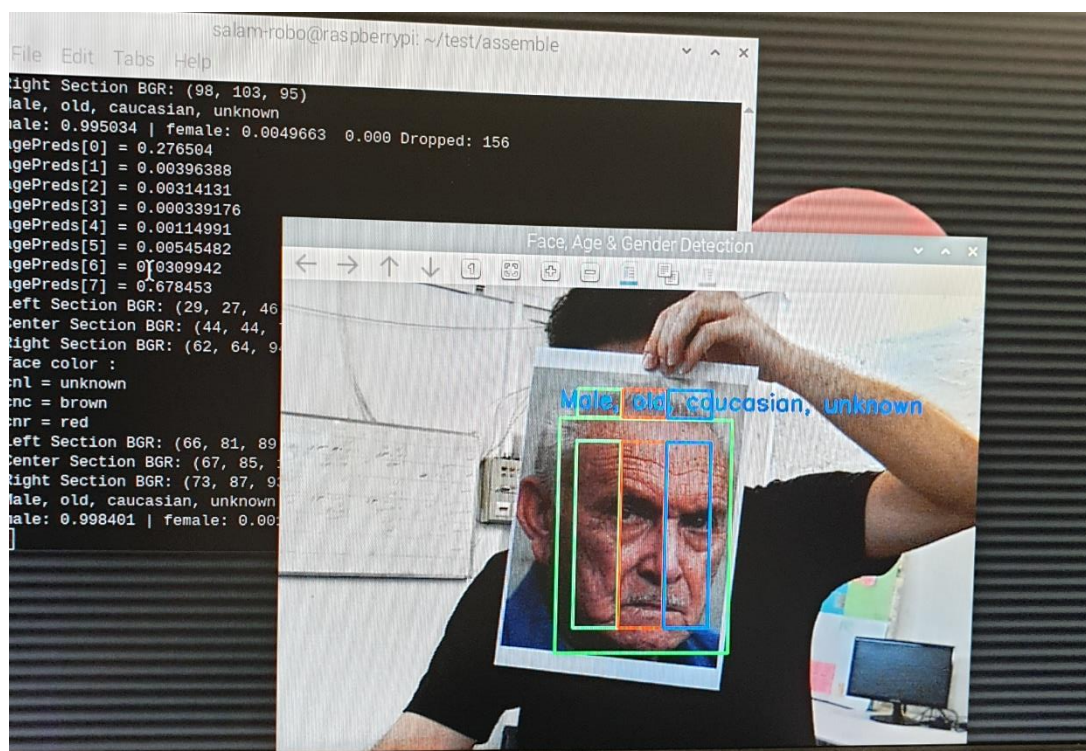
در حالت Face detection، ابتدا باید بررسی کنیم سیگنال موفقیتی که سنسور سونار ارسال کرده مربوط به حضور انسان بوده یا خیر. پس با انجام پردازش های تشخیص چهره، از این موضوع اطمینان حاصل می کنیم. اگر چهره تشخیص داده شود، مدل های پیشرفته تر تشخیص سن، جنسیت و ویژگی های ظاهری دیگر مانند رنگ پوست و نژاد اضافه می شوند تا اطلاعات بیشتری از فرد به دست بیاوریم. در نهایت با اتمام کار مدل ها، اطلاعات بدست آمده را برای پخش ویدیوی شخصی سازی شده آماده می کنیم و وارد حالت Video playback می شویم. همزمان با انجام پردازش های تشخیص چهره، سیگنال های ارسال شده توسط

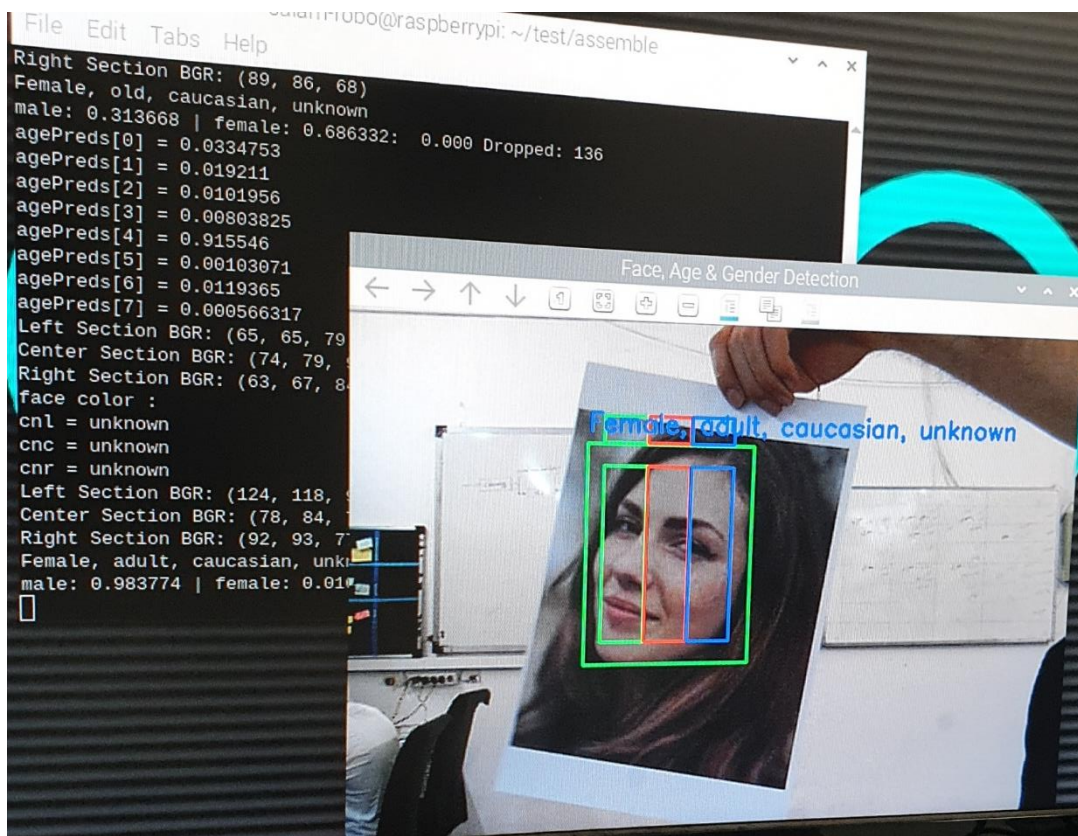
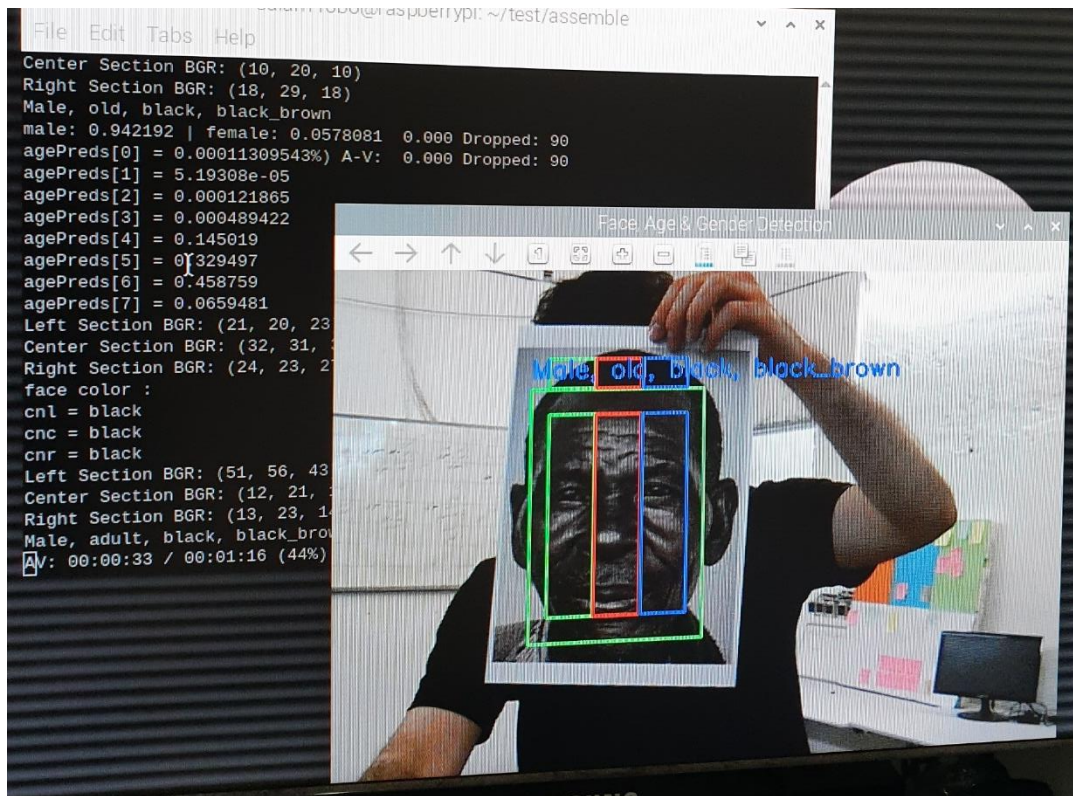
میکروکنترلر پردازش می شوند تا در صورتی که پس از ورود به حالت Face detection، هرگز موفق به تشخیص چهره نشویم، با یکی از دو رویداد زیر به حالت قبلی (Sonar detection) برگردیم:

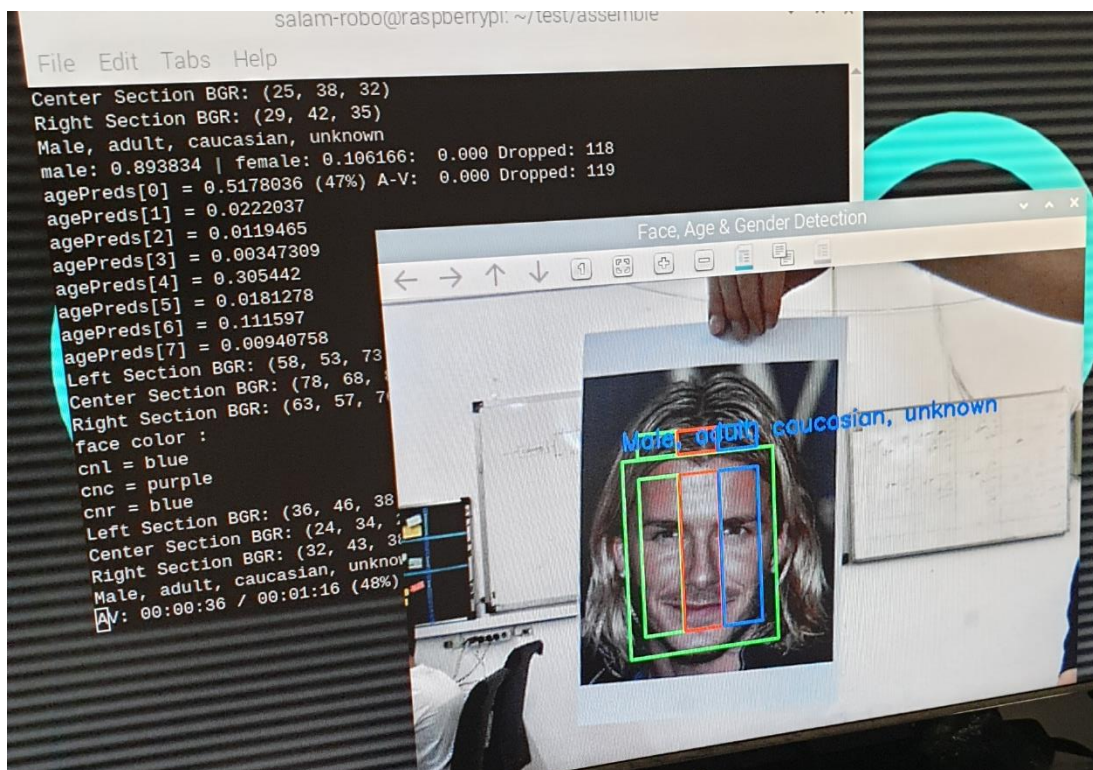
- اگر چند ثانیه از ورود به حالت Face detection گذشته باشد و میکروکنترلر سیگنال شکست را ارسال کند، یعنی فردی در حال حاضر مقابل ربات قرار نگرفته که تشخیص چهره انجام شود.
- اگر زمان مشخصی از ورود به حالت Face detection گذشته باشد و نه ریزپردازنده قابل به تشخیص چهره باشد و نه میکروکنترلر سیگنال شکست را ارسال کند، یعنی احتمالا جسم غیر انسان مقابل سنسور سونار قرار گرفته و باید به حالت قبل برگردیم تا از پردازش اضافی جلوگیری کنیم. به این زمان مشخص watchdog timeout گفته می شود و از لحظه ورود به هر حالت reset و محاسبه می گردد.

در واقع اگر در Face detection در ابتدا موفق به تشخیص چهره نشویم، اتفاقی نمی افتد و در همان حالت باقی می مانیم، مگر اینکه یکی از دو رویداد بالا رخ دهد تا به حالت قبلی برویم یا پیش از رخ دادن رویداد های بالا موفق به تشخیص چهره شویم تا به حالت بعدی برویم.

عکس های زیر چهره های تشخیص داده شده به همراه ویژگی های ظاهری آن ها را نمایش می دهند:







همانطور که در عکس های بالا مشخص است، چهره افراد به همراه ویژگی های ظاهریشان تشخیص داده شده است. در نمونه های بالا، ویژگی های ظاهری نوشته شده در تصاویر به ترتیب عبارتند از: جنسیت، سن و نژاد.

Video playback state

در حالت Video playback، اطلاعات استخراج شده از حالت Face detection برای پخش ویدئوی تبلیغ استفاده می شود. این حالت تا زمانی ادامه پیدا می کند که ویدئوی تبلیغ تمام شود یا میکروکنترلر سیگنال شکست را به رزبری پای ارسال کند. در صورتی که سیگنال شکست دریافت شود، ویدئو تبلیغ ناتمام قطع می شود. به هر صورت در پایان این حالت، گیف screen saver مجدداً بر روی صفحه نمایش داده می شود و ربات به حالت Sonar detection باز می گردد.

3-4- اجرای ماشین حالت متناهی با استفاده از چندریسمانی

چندریسمانی به مفهوم اجرای همزمان چندین وظیفه در یک سیستم است که هرکدام در یک رشته (thread) جداگانه اجرا می شود. این قابلیت در سیستم های چندپردازنده ای یا چند هسته ای، عملکرد بهتری از خود نشان می دهد و می تواند به افزایش بهره وری کمک کند.

می دانیم پردازش تصویر، که بر روی رزبری پای انجام می شود، وظیفه ای سنگین از نظر بار پردازشی است و در

ربات تعاملی سلام در همه حالات به جز پخش ویدئو انجام می شود. از طرفی در هر حالت ربات تعاملی شروطنی را بررسی می کند که مستقل از تشخیص چهره یا رنگ لباس هستند و پیچیدگی محاسبات پردازش تصویر می تواند بر سرعت تصمیم گیری در وظایف دیگر تاثیر بگذارد. بنابراین استفاده از یک ریسمان جدا فقط برای پردازش تصاویر و کاهش بار پردازشی از دوش دیگر وظایف حیاتی منطقی است. همچنین اگر میکروکنترلر آردوینو را یک هسته و ریسمان جدا در نظر بگیریم، وظایف اصلی که باید به صورت جداگانه اجرا شوند به صورت زیر است:

- ارسال سیگنال های موفقیت و شکست با استفاده از سنسور سونار (آردوینو)
 - پردازش تصویر و تشخیص چهره یا رنگ لباس با استفاده از دوربین (رزربری پای)
 - بروزرسانی ماشین حالت متناهی با توجه به ورودی ها برای پخش فایل ویدئویی یا صوتی (رزربری پای)
- برای دریافت سیگنال های آردوینو کلاس Sonar، انجام وظایف مربوط به پردازش تصویر کلاس Camera، پخش فایل ویدئویی و صوتی کلاس Monitor و بروزرسانی ماشین حالت متناهی کلاس FSM تعریف می شود.

هماهنگ سازی و همگامی میان ریسمان ها

کلاس های فوق در کد main.cpp به صورت زیر با یکدیگر ترکیب می شوند:

```
#include <thread>
#include <atomic>
#include <chrono>
#include <iostream>

#include "Sonar2.h"
#include "Camera.h"
#include "Monitor.h"
#include "FSM.h"

#define SLEEP_TIME 10

using namespace std;

int main() {
    std::atomic<bool> running(true);

    Sonar2 sonar(17, 27);
    Camera camera;
    Monitor monitor("/home/salam-robo/test/assemble/videos", "/home/salam-robo/test/assemble/audios", "/home/salam-robo/test/assemble/gif3.mp4");
    FSM fsm;
    string gif = "home/salam-robo/test/assemble/gif1.mp4";
```

```

std::thread cameraThread([&]() {
    camera.update();
    cout << "thread cam" << endl;
});

while (running.load()) {
    fsm.update(sonar, camera, monitor);
    monitor.update();
    std::this_thread::sleep_for(std::chrono::milliseconds(SLEEP_TIME));
}

while (true) {
    std::this_thread::sleep_for(std::chrono::seconds(3));
}

cameraThread.join();

return 0;
}

```

در داخل حلقه اصلی برنامه، عملیات به‌روزرسانی کلاس‌های Monitor و FSM به‌طور هم‌زمان انجام می‌شود. یک ریسمان جداگانه برای به‌روزرسانی کلاس Camera به‌طور موازی با سایر بخش‌ها اجرا می‌شود. کلاس Sonar نیز سیگنال‌های ارسال شده توسط آردوینو را مستقل از کد رزبری پای دریافت می‌کند و در اختیار کلاس FSM قرار می‌دهد.

کلاس FSM نیز به‌روزرسانی ماشین حالت‌های تعریف شده را به صورت زیر انجام می‌دهد:

FSM.h :

```

#ifndef FSM_H
#define FSM_H

#include <string>
#include "Sonar2.h"
#include "Camera.h"
#include "Monitor.h"

using namespace std;

class FSM {
public:

```

```

enum State {
    START,
    SONAR_DETECTION,
    FACE_DETECTION,
    PLAY_VIDEO_FILE,
    BYE
};

private:
    State currentState;
    unsigned long stateTimer;
    unsigned long stateStartTimer;

public:
    FSM();

    void setStateMachine(State newState);
    bool watchdog(unsigned int watchdogTimer);
    void stateTimerUpdate();

    void update(const Sonar2& sonar, Camera& camera, Monitor& monitor);

    void printState();
    State getState();
};

// تابع کمکی برای زمان برحسب میلی ثانیه
unsigned long millis();

#endif // FSM_H

```

FSM.cpp :

```

void FSM::update(const Sonar2& sonar, Camera& camera, Monitor& monitor) {
    switch(currentState) {
        case SONAR_DETECTION: {
            if(camera.getSuccess()) {
                cout << camera.getClothColor() << endl;

                cout << "FSM(SONAR_DETECTION): camera.getSuccess()" << endl;

                camera.setTimer();
                camera.setTask(Camera::Task::IDLE);
                camera.setNextTask(Camera::Task::FACE_DETECTION);
            }
        }
    }
}

```

```

        string cd = monitor.getAudiosDirectory() + '/' +
camera.getClothColor() + "/" + camera.getClothColor() + ".mp3";

        monitor.setMode(Monitor::Mode::AUDIO);
        monitor.setStream(cd);
        monitor.setTask(Monitor::Task::PLAY);
    }
    if(sonar.getSuccess()) {

        cout << "FSM(SONAR_DETECTION): sonar.getSuccess()" << endl;

        camera.setMode(Camera::Mode::CLOSE_UP);

        setStateMachine(FACE_DETECTION);
    }
    break;
}
case FACE_DETECTION:
    stateTimerUpdate();
    if(camera.getSuccess()) {

        cout << "FSM(FACE_DETECTION): camera.getSuccess()" << endl;
        cout << camera.getFaceFeatures().gender << " | " <<
camera.getFaceFeatures().age << endl;

        string cd = monitor.getVideosDirectory() + '/' +
camera.getFaceFeatures().gender + '/' + camera.getFaceFeatures().age; //HERE
        cout << cd << endl;

        monitor.setStream(cd);
        monitor.setMode(Monitor::Mode::VIDEO);
        monitor.setTask(Monitor::Task::PLAY);
        setStateMachine(PLAY_VIDEO_FILE);
    } else if((watchdog(5000) && sonar.getFail()) || watchdog(10000)) {
        if(sonar.getFail()) cout << "FSM(FACE_DETECTION):
sonar.getFail()" << endl;
        else cout << "FSM(FACE_DETECTION): watchdog()" << endl;
        camera.setMode(Camera::Mode::DISTANT);

        setStateMachine(SONAR_DETECTION);
    }
    break;

case PLAY_VIDEO_FILE:
    stateTimerUpdate();

```

```

        if((sonar.getFail() && watchdog(3000)) || monitor.getStreamFinished()
|| watchdog(60000)) {
            if(sonar.getFail()) {
                cout << "FSM(PLAY_VIDEO_FILE): sonar.getFail()" << endl;
            }
            monitor.setTask(Monitor::Task::STOP);
            setStateMachine(BYE);
        }
        break;

    case BYE:
        stateTimerUpdate();
        if(watchdog(1000)) {
            setStateMachine(SONAR_DETECTION);
        }
        break;
    }
}

```

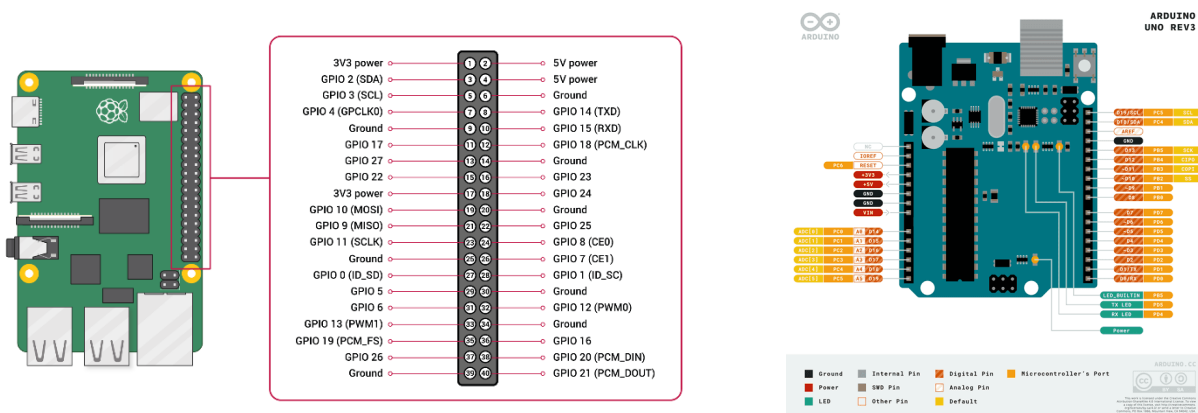
محتویات کلاس FSM در بخش 4-2 کامل و دقیق توضیح داده شد.

فصل پنجم

تشخیص حضور انسان با استفاده از آردوینو و سنسور سونار

5-1- اتصال سونار به آردوینو و آردوینو به رزبری پای

پین های echo و trig سنسور سونار به دو تا از پورت های دیجیتال PWM آردوینو متصل شده اند. همچنین با اتصال دو پورت دیجیتال PWM دیگر به پین های GPIO 17 و GPIO 27 رزبری پای، سیگنال های موفقیت و شکست فرستاده می شوند. Vcc و Gnd آردوینو و سونار نیز با رزبری پای موازی می شوند. نحوه پیدا کردن پورت ها و پین ها در آردوینو Uno R3 و رزبری پای 4 در شکل 5-1 نمایش داده شده است.



شکل 5-1: pinout آردوینو (راست) و رزبری پای (چپ)

5-2- کدنویسی سنسور سونار در آردوینو

کد نوشته شده در آردوینو برای محاسبه فاصله امواج رفت و برگشتی سنسور سونار و ارسال سیگنال های موفقیت و شکست به صورت زیر است:

```
const int trigPin = 9;
const int echoPin = 10;
const int signalPinSuccess = 5;
const int signalPinFail = 6;
const int threshold = 100;
```

```

long duration;
int distance;
int ul_counter_success = 0;
int ul_counter_fail = 0;

void setup() {
    Serial.begin(9600);
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
    pinMode(signalPinSuccess, OUTPUT);
    pinMode(signalPinFail, OUTPUT);

    digitalWrite(signalPinSuccess, LOW);
    digitalWrite(signalPinFail, LOW);

    Serial.println("سنسور آماده است");
}

void loop() {
    // خواندن فاصله
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    duration = pulseIn(echoPin, HIGH);
    distance = duration * 0.034 / 2;

    Serial.print("فاصله: ");
    Serial.print(distance);
    Serial.println(" cm");

    if (distance > 0 && distance <= threshold) {
        ul_counter_success++;
        ul_counter_fail = 0;

        if (ul_counter_success == 10) {
            sendPulse(signalPinSuccess);
            Serial.println("success");
            ul_counter_success = 0;
        }
    }
    else if (distance > threshold || distance == 0) {
        ul_counter_fail++;
    }
}

```

```

    ul_counter_success = 0;

    if (ul_counter_fail == 10) {
        sendPulse(signalPinFail);
        Serial.println("failure");
        ul_counter_fail = 0;
    }
}
else {
    ul_counter_success = 0;
    ul_counter_fail = 0;
}

delay(100); // کوتاه تر برای پاسخگویی بهتر
}

void sendPulse(int pin) {
    digitalWrite(pin, HIGH);
    delay(50);
    digitalWrite(pin, LOW);
}

```

در این برنامه، پین‌های trigPin و echoPin برای ارسال و دریافت سیگنال اولتراسونیک استفاده می‌شوند و فاصله بین سنسور و یک جسم با محاسبه مدت زمان بازگشت سیگنال (که در متغیر duration ذخیره می‌شود) به دست می‌آید. فاصله محاسبه شده در متغیر distance ذخیره می‌شود و اگر این فاصله کمتر از مقدار تعیین شده در threshold باشد، شمارشگر موفقیت (ul_counter_success) افزایش می‌یابد. پس از 10 بار موفقیت متوالی، پالس از پین signalPinSuccess ارسال می‌شود و پیامی با عبارت "success" در سریال مانیتور چاپ می‌شود. اگر فاصله بیشتر از threshold باشد یا سیگنال از دست رفته باشد (یعنی فاصله صفر)، شمارشگر شکست (ul_counter_fail) افزایش می‌یابد و پس از 10 بار شکست متوالی، پالس از پین signalPinFail ارسال می‌شود و پیامی با عبارت "failure" چاپ می‌شود. همچنین، در هر دور از حلقه، تأخیر کوتاهی (100 میلی ثانیه) برای بهبود عملکرد و پاسخگویی سیستم لحاظ شده است. در انتهای هر ارزیابی، تابع sendPulse برای ارسال پالس کوتاه 50 میلی ثانیه ای از پین‌های سیگنال استفاده می‌شود.

5-3- کلاس Sonar در رزبری پای

با ارسال سیگنال‌های موفقیت و شکست به رزبری پای، باید آن‌ها را در دریافت کرده و به داده‌های قابل استفاده تبدیل کنیم. این کار در کلاس Sonar به صورت زیر انجام شده است:

Sonar2.h :

```
#ifndef SONAR2_H
#define SONAR2_H

using namespace std;

class Sonar2 {
private:
    int pinSuccess;
    int pinFail;

public:
    Sonar2(int successPin, int failPin);
    bool getSuccess() const;
    bool getFail() const;
};

#endif // SONAR2_H
```

Sonar2.cpp :

```
#include <wiringPi.h>
#include <iostream>
#include "Sonar2.h"

using namespace std;

Sonar2::Sonar2(int successPin, int failPin)
    : pinSuccess(successPin), pinFail(failPin)
{
    wiringPiSetupGpio(); // استفاده از شماره گذاری BCM
    pinMode(pinSuccess, INPUT);
    pinMode(pinFail, INPUT);
    cout << "Sonar2 is constructed" << endl;
}

bool Sonar2::getSuccess() const {
    return digitalRead(pinSuccess) == HIGH;
}

bool Sonar2::getFail() const {
    return digitalRead(pinFail) == HIGH;
}
```

فصل ششم

تشخیص چهره و لباس

6-1- تنظیم رنگ و پردازش فریم

هدف از تنظیم رنگ (Color calibration)، تصحیح انحرافات رنگی ناشی از عواملی مانند نور محیط، ویژگی‌های دوربین، یا نوع سنسور است. بدون این تصحیح، رنگ‌ها در تصاویر ممکن است به‌طور غیرواقعی به‌نظر برسند یا دچار تغییرات ناخواسته شوند که منجر به نتایج نادرست در پردازش تصویر می‌شود.

استفاده از مدل‌های رنگی استاندارد مانند RGB (قرمز، سبز، آبی)، HSV (رنگ، اشباع، روشنایی)، و ... به تنظیم دقیق‌تر رنگ‌ها کمک می‌کند. این مدل‌ها معمولاً می‌توانند رنگ‌ها را به‌گونه‌ای تبدیل کنند که وابستگی به تنظیمات دوربین و نور را کاهش دهند.

BGR (Blue, Green, Red) و HSV (Hue, Saturation, Value) دو مدل رنگی مختلف هستند که برای نمایش رنگ‌ها در پردازش تصویر استفاده می‌شوند. در مدل BGR، رنگ‌ها به‌طور مستقیم از سه کانال رنگی اصلی آبی، سبز و قرمز ترکیب می‌شوند و این مدل بیشتر در نمایشگرها و پردازش‌های دیجیتال استفاده می‌شود. در مقابل، مدل HSV رنگ‌ها را بر اساس سه ویژگی اصلی تعریف می‌کند: Hue (رنگ اصلی)، Saturation (شدت رنگ) و Value (روشنایی یا مقدار رنگ). مدل HSV بیشتر در شناسایی و پردازش ویژگی‌های رنگی به‌ویژه در کاربردهای تصویربرداری و شناسایی اشیاء مفید است زیرا قابلیت تفکیک بهتر رنگ‌ها را فراهم می‌آورد و شباهت‌های رنگی را نسبت به BGR دقیق‌تر می‌سازد.

در ادامه متد‌های استفاده شده برای تشخیص و تنظیم رنگ‌ها در کلاس Camera آمده است:

```
cv::Vec3b Camera::calculateBGRMean(const cv::Mat& image, const cv::Rect& roi) {
    // Crop the region of interest (ROI) from the image
    cv::Mat roiImage = image(roi);

    // Calculate the mean of the BGR values
    cv::Scalar meanScalar = cv::mean(roiImage);

    // Convert the mean value to Vec3b for BGR
    cv::Vec3b meanBGR(static_cast<uchar>(meanScalar[0]),
```

```

        static_cast<uchar>(meanScalar[1]),
        static_cast<uchar>(meanScalar[2]));

    return meanBGR;
}

std::tuple<std::string, std::string, std::string> Camera::processFrame(cv::Mat&
frame, cv::Rect roi) {
    // به سه بخش ROI تقسیم

    int roiWidth = roi.width / 3;
    cv::Rect roiLeft(roi.x, roi.y, roiWidth, roi.height); // بخش چپ
    cv::Rect roiCenter(roi.x + roiWidth, roi.y, roiWidth, roi.height); // بخش
    وسط
    cv::Rect roiRight(roi.x + 2 * roiWidth, roi.y, roiWidth, roi.height); //
    بخش راست

    try {
        // برای هر بخش BGR محاسبه میانگین
        cv::Vec3b avgBGRLeft = calculateBGRMean(frame, roiLeft);
        cv::Vec3b avgBGRCenter = calculateBGRMean(frame, roiCenter);
        cv::Vec3b avgBGRRight = calculateBGRMean(frame, roiRight);

        // کشیدن مستطیل برای هر بخش
        cv::rectangle(frame, roiLeft, cv::Scalar(0, 255, 0), 2); // مستطیل برای بخش
        چپ
        cv::rectangle(frame, roiCenter, cv::Scalar(0, 0, 255), 2); // مستطیل برای
        بخش وسط
        cv::rectangle(frame, roiRight, cv::Scalar(255, 0, 0), 2); // مستطیل برای
        بخش راست

        // برای هر بخش BGR نمایش میانگین رنگ
        std::cout << "Left Section BGR: ("
            << static_cast<int>(avgBGRLeft[0]) << ", "
            << static_cast<int>(avgBGRLeft[1]) << ", "
            << static_cast<int>(avgBGRLeft[2]) << ")" << std::endl;

        std::cout << "Center Section BGR: ("
            << static_cast<int>(avgBGRCenter[0]) << ", "
            << static_cast<int>(avgBGRCenter[1]) << ", "
            << static_cast<int>(avgBGRCenter[2]) << ")" << std::endl;

        std::cout << "Right Section BGR: ("
            << static_cast<int>(avgBGRRight[0]) << ", "

```

```

        << static_cast<int>(avgBGRRight[1]) << ", "
        << static_cast<int>(avgBGRRight[2]) << ")" << std::endl;

    // دریافت رنگ شناسایی شده با استفاده از متد classifyColor
    std::string colorNameLeft = classifyColor(avgBGRLeft); // گرفتن اسم رنگ برای
بخش چپ
    std::string colorNameCenter = classifyColor(avgBGRCenter); // گرفتن اسم رنگ
برای بخش وسط
    std::string colorNameRight = classifyColor(avgBGRRight); // گرفتن اسم رنگ
برای بخش راست

    // نمایش اسم رنگ ها
    // std::cout << "Left Section Color: " << colorNameLeft << std::endl;
    // std::cout << "Center Section Color: " << colorNameCenter << std::endl;
    // std::cout << "Right Section Color: " << colorNameRight << std::endl;

    // برگرداندن میانگین رنگ ها از سه بخش
    return std::make_tuple(colorNameLeft, colorNameCenter, colorNameRight);
} catch (const cv::Exception& e) {
    return std::make_tuple("unknown", "unknown", "unknown");
}
}

std::string Camera::classifyColor(cv::Vec3b bgr) { //new HERE tabestoon

    int b = bgr[0]; // Blue component
    int g = bgr[1]; // Green component
    int r = bgr[2]; // Red component

    // تبدیل BGR به HSV
    Mat bgrMat(1, 1, CV_8UC3, Scalar(b, g, r)); // BGR در OpenCV
    Mat hsvMat;
    cvtColor(bgrMat, hsvMat, COLOR_BGR2HSV); // تبدیل BGR به HSV

    // استخراج مقادیر HSV
    Vec3b hsv = hsvMat.at<Vec3b>(0, 0);
    int h = hsv[0]; // Hue
    int s = hsv[1]; // Saturation
    int v = hsv[2]; // Value

    int max_val = max({r, g, b}); // بیشترین مقدار بین r، g و b
    int min_val = min({r, g, b}); // کمترین مقدار بین r، g و b
    int mid_val = r + g + b - max_val - min_val;

    // محاسبه اختلاف max و min

```

```

int maxDiff = max_val - min_val; // اختلاف بین max و min
int upDiff = max_val - mid_val;
int downDiff = mid_val - min_val;

if((max_val < 20) || (maxDiff < 10 && max_val <= 100)) { //max_val < 50
    return "black";
}

if (v > 90 && s < 10) { // کم باشد Saturation زیاد باشد و Value اگر
    return "white";
}

if(max_val > 90 && upDiff > 20 && maxDiff > 20) { //(max_val > 130 && upDiff
> 20) ||
    if(max_val == r) {
        return "red";
    }
    if(max_val == b) {
        return "blue";
    }
    if(max_val == g) {
        return "green";
    }
}

if(max_val <= 100) {
    if(max_val == r && mid_val == g && upDiff > 20) {
        return "brown";
    }
    if(max_val == g && upDiff > 20) {
        return "green";
    }
}

if(upDiff < 30 && maxDiff > 2*upDiff) {
    if(min_val == g) {
        if((max_val > 170 && max_val == r) || r > 210) {
            return "pink";
        }
        return "purple";
    }
}

if(min_val == b) {

```

```

        if((max_val == r && r > 170) || (max_val == g && g > 230)) {
            return "yellow";
        }
    }
    if(min_val == r && maxDiff > 80) {
        if(max_val > 160) {
            return "cyan";
        }
    }
}

if(downDiff < 20) {
    if(max_val == b && b > 140 && upDiff > 100) {
        return "blue";
    }
    if(max_val == r && r < 120 && upDiff > 70) {
        //nmd;
    }
}
//
if(max_val == r) {
    if(upDiff > 130) {
        return "red";
    }
    if(upDiff < 20 && downDiff > 60 && r > 170) {
        if(mid_val == g) {
            return "yellow";
        }
    }
}

// تشخیص رنگ سیاه (dark color)
if (r < 30 && g < 30 && b < 30) {
    return "black";
}

// تشخیص رنگ سفید (light color)
if (r > 140 && g > 140 && b > 140) {
    return "white";
}

// تشخیص خاکی یا خاکستری
if (r > 100 && g > 100 && b > 100 && r < 150 && g < 150 && b < 150) {
    return "gray";
}

```

```

}

// دارد R قرمز: رنگ قرمز معمولاً بیشترین مقدار را در
if ((r > 120 && g < 80 && b < 80) || (b < 10 && g < 10 && r > 30)) {
    return "red";
}

// دارد G سبز: رنگ سبز معمولاً بیشترین مقدار را در
if (g > 120 && r < 80 && b < 80) {
    return "green";
}

// دارد B آبی: رنگ آبی معمولاً بیشترین مقدار را در
if (b > 120 && r < 80 && g < 80) {
    return "blue";
}

// نارنجی: ترکیب زیاد قرمز و کمی سبز، کم بودن آبی
if (r > 130 && g > 100 && b < 80) {
    return "orange";
}

// زرد: ترکیب زیاد قرمز و سبز، کم بودن آبی
if (r > 100 && g > 100 && b < 80) {
    return "yellow";
}

// بنفش: ترکیب زیاد قرمز و آبی
if (r > 100 && b > 100 && g < 80) {
    return "purple";
}

// صورتی: ترکیب زیاد قرمز و آبی، کمی سبز
if (r > 180 && g < 100 && b > 150) {
    return "pink";
}

// HSV

// بر اساس (brown) تشخیص رنگ قهوه‌ای
if (h >= 10 && h < 30 && s > 40 && v > 100 && v < 150) {
    return "brown";
}

// Hue بر اساس (red) تشخیص رنگ قرمز

```

```

    if (h >= 0 && h < 10 && s > 100) {
        return "red";
    }

    // Hue بر اساس (green) تشخیص رنگ سبز
    if (h >= 35 && h < 85 && s > 40) {
        return "green";
    }

    // Hue بر اساس (blue) تشخیص رنگ آبی
    if (h >= 85 && h < 175 && s > 40) {
        return "blue";
    }

    // تشخیص رنگ نارنجی (orange)
    if (h >= 10 && h < 35 && s > 100 && v >= 150) {
        return "orange";
    }

    // تشخیص رنگ زرد (yellow)
    if (h >= 35 && h < 65 && s > 100) {
        return "yellow";
    }

    // تشخیص رنگ بنفش (purple)
    if (h >= 125 && h < 175 && s > 40) {
        return "purple";
    }

    // صورتی (pink)
    if (h >= 160 && h < 180 && s > 100 && v > 180) {
        return "pink";
    }

    // برگردانده می شود "unknown" اگر هیچکدام از شرایط رنگ های اصلی و فرعی صدق نکرد،
    return "unknown";
}

bool Camera::isDominantColor(cv::Mat& frame, Rect roi, float thresh) { //new HERE
    tabestoon : sum up all pixels at bgr, scale the summed result to 0-255 by
    subtraction and apply classifyColor to the result. batch process
    if (frame.empty() || roi.x < 0 || roi.y < 0 || ((roi.x + roi.width) >
    frame.cols) || ((roi.y + roi.height) > frame.rows) || roi.width <= 0 ||
    roi.height <= 0)

```

```

        return false;

    // Rect roi(x1, y1, dx, dy);
    Mat region = frame(roi);
    if(region.empty())
        return false;

    map<string, int> colorCounts;
    int totalPixels = region.rows * region.cols;

    // شمارش تعداد پیکسل‌ها در هر دسته رنگ
    for (int y = 0; y < region.rows; ++y) {
        for (int x = 0; x < region.cols; ++x) {
            Vec3b pixel = region.at<Vec3b>(y, x);
            string colorName = classifyColor(pixel);
            colorCounts[colorName]++;
        }
    }

    // پیدا کردن رنگ غالب
    int maxCount = 0;
    for (auto& pair : colorCounts) {
        if (pair.second > maxCount) {
            maxCount = pair.second;
            this->dominantColor = pair.first;
        }
    }

    double ratio = static_cast<double>(maxCount) / totalPixels;
    return ratio >= thresh;
}

```

- **calculateBGRMean**: این متد میانگین رنگ‌های BGR یک بخش مشخص از تصویر (محدوده منطقه‌ای یا ROI) را محاسبه می‌کند. ابتدا منطقه مشخص شده از تصویر برش داده می‌شود، سپس میانگین مقادیر رنگ BGR آن محاسبه و به صورت یک `cv::Vec3b` (رنگ در فضای BGR) بازگشت داده می‌شود.
- **processFrame**: این متد تصویر ورودی را به سه بخش (چپ، وسط، راست) تقسیم می‌کند و برای هر بخش، میانگین رنگ BGR را با استفاده از متد `calculateBGRMean` محاسبه می‌کند. سپس، مستطیل‌هایی برای هر بخش ترسیم می‌کند و نام رنگ‌ها را با استفاده از متد `classifyColor` برای هر بخش استخراج کرده و در قالب یک `tuple` باز می‌گرداند.

- **classifyColor**: این متد رنگ ورودی (در قالب cv::Vec3b) را با تبدیل هر دو به فضای رنگ BGR و HSV تجزیه و تحلیل می‌کند و رنگ آن را طبق ویژگی‌های Hue، Saturation و Value یا Blue، Green و Red شناسایی می‌کند. این متد انواع رنگ‌ها مانند قرمز، سبز، آبی، زرد، سفید، مشکی و دیگر رنگ‌ها را بر اساس مقادیر مختلف HSV و BGR شناسایی می‌کند.
- **isDominantColor**: این متد بررسی می‌کند که آیا رنگ غالب در یک بخش مشخص از تصویر (ROI) بر اساس یک آستانه (threshold) مشخص بیشتر از حد تعیین شده است یا خیر. ابتدا رنگ‌ها برای هر پیکسل در ROI با استفاده از متد classifyColor شناسایی می‌شوند، سپس تعداد هر رنگ شمرده می‌شود و رنگ غالب شناسایی می‌شود. در نهایت، نسبت پیکسل‌های رنگ غالب به کل پیکسل‌ها محاسبه شده و در صورت برآورده شدن آستانه، true بازگشت داده می‌شود.

6-2- ساختار کلاس Camera

در هدر کلاس Camera دو مفهوم اساسی تعریف شده است: mode و task.

```
enum Task {
    TURN_ON,
    FACE_DETECTION,
    TURN_OFF,
    IDLE
};
enum Mode {
    CLOSE_UP,
    DISTANT
};
```

تمام وظایف مربوط به تشخیص چهره و استفاده از مدل‌های یادگیری ماشین در تسک FACE_DETECTION رخ می‌دهد. این تسک ارتباطی با حالت Face detection تعریف شده در کلاس FSM ندارد. زمانی که در حالت Video playback یا در حال پخش صوت در Sonar detection هستیم، تسک IDLE انتخاب می‌شود.

مد‌های CLOSE_UP و DISTANT برای تفاوت قائل شدن میان الگوریتم‌های تشخیص سن و جنسیت در Face detection با الگوریتم‌های تشخیص رنگ لباس در Sonar detection می‌باشند.

همچنین در هدر کلاس Camera، خصوصیات چهره تشخیص داده شده به صورت زیر دسته بندی می‌شوند:

```
typedef struct {
    std::string gender;      // male, female
    std::string age;        // kid, adult, old
    std::string race;       // black, asian, caucasian
    std::string hairColor;  // blonde, ginger, black_brown, gray_white
} FaceFeatures;
```

مقادیر مجاز برای مقداردهی این متغیرها که در کامنت‌های رو به روی آن‌ها نوشته شده است. دلیل تعریف این متغیرها از نوع رشته، مقداردهی به دیرکتوری ویدئوی مطلوب در کلاس Monitor است.

در constructor کلاس Camera نیز مدل‌های شبکه عصبی عمیق به صورت زیر تعریف می‌شوند:

```
Camera::Camera() : currentTask(IDLE) {
    std::string faceProto = "face_detector/opencv_face_detector.pbtxt";
    std::string faceModel = "face_detector/opencv_face_detector_uint8.pb";
    std::string ageProto = "age_detector/deploy_age.prototxt";
    std::string ageModel = "age_detector/age_net.caffemodel";
    std::string genderProto = "gender_detector/deploy_gender.prototxt";
    std::string genderModel = "gender_detector/gender_net.caffemodel";
    facemark = FacemarkLBF::create();
    facemark->loadModel("lbfmodel.yaml");
    faceNet = dnn::readNetFromTensorflow(faceModel, faceProto);
    ageNet = dnn::readNetFromCaffe(ageProto, ageModel);
    genderNet = dnn::readNetFromCaffe(genderProto, genderModel);

    this->ageList = { "(0-2)", "(4-6)", "(8-12)", "(15-20)",
                     "(25-32)", "(38-43)", "(48-53)", "(60-100)" };
    this->genderList = { "Male", "Female" };
    this->currentMode = DISTANT;

    this->ComposureCounter = 1;
    this->setTask(TURN_ON);

    cout << "Camera constructed" << endl;
}
```

6-2-1- کدنویسی تشخیص چهره

در متد update کلاس Camera:

```
case FACE_DETECTION: {
    if (!cap.isOpened()) {
        cerr << "Camera is not opened!\n";
        break;
    }
    Mat frame;
    cap >> frame;

    if (frame.empty()) {
        cerr << "Blank frame grabbed\n";
        break;
    }

    // تشخیص چهره
    Mat blob = dnn::blobFromImage(frame, 1.0, Size(300, 300),
                                   Scalar(104.0, 177.0, 123.0), true,
false);

    faceNet.setInput(blob);
    Mat detections = faceNet.forward();
    Mat detectionMat(detections.size[2], detections.size[3], CV_32F,
detections.ptr<float>());

    for (int i = 0; i < detectionMat.rows; i++) {
        float confidence = detectionMat.at<float>(i, 2);

        if (confidence > CONFIDENCE_THRESHOLD &&
!((ComposureCounter++)%1)) {

            int x1 = static_cast<int>(detectionMat.at<float>(i, 3) *
frame.cols);
            int y1 = static_cast<int>(detectionMat.at<float>(i, 4) *
frame.rows);
            int x2 = static_cast<int>(detectionMat.at<float>(i, 5) *
frame.cols);
            int y2 = static_cast<int>(detectionMat.at<float>(i, 6) *
frame.rows);

            x1 = max(0, x1);
            y1 = max(0, y1);
            x2 = min(frame.cols - 1, x2);
            y2 = min(frame.rows - 1, y2);
```

```
Rect faceBox(Point(x1, y1), Point(x2, y2));
rectangle(frame, faceBox, Scalar(0, 255, 0), 2);

Mat face = frame(faceBox);
```

$x1, y1, x2, y2$ مختصات کوچک ترین مربعیست که چهره تشخیص داده شده در آن است.

6-2-2- کد نویسی تشخیص رنگ لباس

در ادامه کد بالا، زمانی که شرط Camera::Mode برای DISTANT یا CLOSE_UP بودن بررسی می شود، اگر مد بر روی DISTANT باشد کد زیر اجرا خواهد شد:

```
Rect roiCloth(1.5*x1 - 0.5*x2, 1.5*y2 - 0.5*y1, 2*(x2 - x1), (frame.rows-
y2)*ROI_RATIO);
    auto [colorClothLeft, colorClothCenter, colorClothRight] =
processFrame(frame, roiCloth);
    int countBlackCloth = 0;
    int countBlueCloth = 0;
    int countRedCloth = 0;
    int countYellowCloth = 0;
    int countGreenCloth = 0;
    int countWhiteCloth = 0;

    if (colorClothLeft == "black") countBlackCloth++;
    if (colorClothCenter == "black") countBlackCloth++;
    if (colorClothRight == "black") countBlackCloth++;

    if (colorClothLeft == "blue") countBlueCloth++;
    if (colorClothCenter == "blue") countBlueCloth++;
    if (colorClothRight == "blue") countBlueCloth++;

    if (colorClothLeft == "red") countRedCloth++;
    if (colorClothCenter == "red") countRedCloth++;
    if (colorClothRight == "red") countRedCloth++;

    if (colorClothLeft == "yellow") countYellowCloth++;
    if (colorClothCenter == "yellow") countYellowCloth++;
    if (colorClothRight == "yellow") countYellowCloth++;

    if (colorClothLeft == "green") countGreenCloth++;
    if (colorClothCenter == "green") countGreenCloth++;
    if (colorClothRight == "green") countGreenCloth++;
```

```

if (colorClothLeft == "white") countWhiteCloth++;
if (colorClothCenter == "white") countWhiteCloth++;
if (colorClothRight == "white") countWhiteCloth++;

if(countBlackCloth > 2) {
    this->setClothColor("black");
    this->success = true;
    // break;
} else if(countBlueCloth > 2) {
    this->setClothColor("blue");
    this->success = true;
    // break;
} else if(countRedCloth > 2) {
    this->setClothColor("red");
    this->success = true;
    // break;
} else if(countYellowCloth > 2) {
    this->setClothColor("yellow");
    this->success = true;
    // break;
} else if(countGreenCloth > 2) {
    this->setClothColor("green");
    this->success = true;
    // break;
} else if(countWhiteCloth > 2) {
    this->setClothColor("white");
    this->success = true;
    // break;
} else {
    this->setClothColor("unknown");
    this->success = true;
    // break;
}

// cout << this->getClothColor() << endl;
// متن روی تصویر
string label = this->clothColor;
cout << label << endl;
putText(frame, label, Point(x1, y1 - 10),
        FONT_HERSHEY_SIMPLEX, 0.7, Scalar(255, 0, 0), 2);
}

```

بنابراین موقعیت ناحیه لباس با توجه به موقعیت ناحیه سر (مختصات چهره تشخیص داده شده) تعیین می شود و سپس در متد processFrame ناحیه لباس به سه بخش تقسیم می شود. برای تشخیص یک رنگ به عنوان رنگ لباس، باید هر سه ناحیه تقسیم بندی شده یک رنگ را تشخیص داده باشند.

6-2-3- اجرای مدل های تشخیص سن و جنسیت

اگر Camera::Mode مقدار CLOSE_UP را داشته باشد، کد زیر اجرا می شود:

```
// cout << "in close-up" << endl;
Mat faceBlob = dnn::blobFromImage(face, 1.0, Size(227, 227),
                                   Scalar(78.4263377603, 87.7689143744,
114.895847746),
                                   false);

// تشخیص جنسیت
genderNet.setInput(faceBlob);
Mat genderPreds = genderNet.forward();
cout << "male: " << genderPreds.at<float>(0) << " | female: " <<
genderPreds.at<float>(1) << endl;
int genderIdx = genderPreds.at<float>(1) >= 0.6 ? 1 : 0;
//genderPreds.at<float>(1) ? 0 : 1;
this->faceFeatures.gender = genderList[genderIdx];

// تشخیص سن
ageNet.setInput(faceBlob);
Mat agePreds = ageNet.forward();
double ageConf;
Point classNumber;

for (int i = 0; i < 8; i++) {
    std::cout << "agePreds[" << i << "] = " << agePreds.at<float>(i) <<
std::endl;
}
if(agePreds.at<float>(0) > 0.3) {
    this->faceFeatures.age = "adult";
} else if(agePreds.at<float>(4) + agePreds.at<float>(5) + agePreds.at<float>(6) +
agePreds.at<float>(7) >
    agePreds.at<float>(1) + agePreds.at<float>(2) + agePreds.at<float>(3))
{ //0.5) {
    if((agePreds.at<float>(3) + agePreds.at<float>(4) + agePreds.at<float>(5)) >
0.4) {
        this->faceFeatures.age = "adult";
    } else if((agePreds.at<float>(6) + agePreds.at<float>(7)) >
    (agePreds.at<float>(3) + agePreds.at<float>(4) + agePreds.at<float>(5)))
{ //oldPreds/summedPreds > 0.6) {
    this->faceFeatures.age = "old";
    } else {
        this->faceFeatures.age = "adult";
    }
}
```

```

} else {
    bool probablyKid = false;
    if(faceBox.y < frame.rows/2) {
        probablyKid = true;
    }
    float summedPreds = agePreds.at<float>(0) + agePreds.at<float>(1) +
agePreds.at<float>(2)
        + agePreds.at<float>(3) + agePreds.at<float>(4) +
agePreds.at<float>(5);
    // float kidPreds = agePreds.at<float>(0) + agePreds.at<float>(1); //+
agePreds.at<float>(2);
    if(((agePreds.at<float>(1)/summedPreds > 0.3 ||
agePreds.at<float>(2)/summedPreds > 0.3) && probablyKid)
        || (agePreds.at<float>(0) + agePreds.at<float>(1))/summedPreds > 0.6) {
//oldPreds/summedPreds > 0.6) {
        this->faceFeatures.age = "kid";
    } else {
        this->faceFeatures.age = "adult";
    }
}

Rect roiFace(x1+(x2-x1)/10, y1+(y2-y1)/10, 4*(x2-x1)/5, 4*(y2-y1)/5); //HERE
tabestoon 8/10

// پردازش فریم‌ها از طریق متد processFrame
auto [colorNameLeft, colorNameCenter, colorNameRight] = processFrame(frame,
roiFace);
cout << "face color : \ncl = " << colorNameLeft << "\ncnc = " << colorNameCenter
<< "\ncnr = " << colorNameRight << endl;
int countBlack = 0;
if(colorNameLeft == "black") { // || colorNameLeft == "brown" || colorNameLeft ==
"red") {
    countBlack++;
}
if(colorNameCenter == "black") { // || colorNameCenter == "brown" ||
colorNameCenter == "red") {
    countBlack++;
}
if(colorNameRight == "black") { // || colorNameRight == "brown" || colorNameRight
== "red") {
    countBlack++;
}

if(countBlack > 2) {
    this->faceFeatures.race = "black";
}

```

```

} else {
    this->faceFeatures.race = "caucasian";
}

// تشخيص موها
int hairlen = (y2-y1)/8; // /6
Rect roiHair(x1+(x2-x1)/10, y1-hairlen, 4*(x2-x1)/5, hairlen);
// rectangle(frame, roiHair, Scalar(255, 0, 0), 2); //HERE tabestoon
auto [colorHairLeft, colorHairCenter, colorHairRight] = processFrame(frame,
roiHair);
int countYellowHair = 0;
int countBlackHair = 0;
int countGrayHair = 0;
int countOrangeHair = 0;

if(colorHairLeft == "yellow") {
    countYellowHair++;
}
if(colorHairCenter == "yellow") {
    countYellowHair++;
}
if(colorHairRight == "yellow") {
    countYellowHair++;
}

if(colorHairLeft == "black" || colorHairLeft == "brown") {
    countBlackHair++;
}
if(colorHairCenter == "black" || colorHairCenter == "brown") {
    countBlackHair++;
}
if(colorHairRight == "black" || colorHairRight == "brown") {
    countBlackHair++;
}

if(colorHairLeft == "gray" || colorHairLeft == "white") {
    countGrayHair++;
}
if(colorHairCenter == "gray" || colorHairCenter == "white") {
    countGrayHair++;
}
if(colorHairRight == "gray" || colorHairRight == "white") {
    countGrayHair++;
}
}

```

```

if(colorHairLeft == "red" || colorHairLeft == "orange") {
    countOrangeHair++;
}
if(colorHairCenter == "red" || colorHairCenter == "orange") {
    countOrangeHair++;
}
if(colorHairRight == "red" || colorHairRight == "orange") {
    countOrangeHair++;
}

if(countYellowHair > 0) {
    this->faceFeatures.hairColor = "blonde";
} else if(countOrangeHair > 0) {
    this->faceFeatures.hairColor = "ginger";
} else if(countBlackHair > 1) {
    this->faceFeatures.hairColor = "black_brown";
} else if(countGrayHair > 1) {
    this->faceFeatures.hairColor = "gray_white";
} else {
    this->faceFeatures.hairColor = "unknown";
}

// متن روی تصویر
string label = this->faceFeatures.gender + ", " + this->faceFeatures.age + ", " +
this->faceFeatures.race
                + ", " + this->faceFeatures.hairColor;
cout << label << endl;
putText(frame, label, Point(x1, y1 - 10),
        FONT_HERSHEY_SIMPLEX, 0.7, Scalar(255, 0, 0), 2);
this->success = true;

```

که ترکیبی از مدل های شبکه های عصبی عمیق و کدنویسی کلاسیک است.

3-6- استفاده از Tensorflow C++ api برای آموزش مدل های دقیق تر

نتایج بدست آمده از مدل های شبکه عصبی عمیق برای تشخیص سن و جنسیت همیشه مطلوب نبودند، بنابراین باید به دنبال راه حل جایگزین برای مدل های دانلود شده از اینترنت می گشتم. اولین ایده ای که به ذهن می رسید آموزش مدل های تشخیص سن و جنسیت با استفاده از Tensorflow C++ api بود.

3-6-1- دانلود دیتاست و نرمالسازی با تشخیص چهره

یک دیتاست شامل بیش از 24 هزار عکس انسان با لیبل های {سن، جنسیت، نژاد} از سایت Kaggle دانلود شده است. برای آموزش مدل و استفاده در کد اصلی، ابتدا باید داده ها را نرمالسازی کنیم. در مورد ربات تعاملی

سلام، نرمالسازی به تشخیص چهره ها و `resize` کردن تصاویر دیتاست محدود می شود. پس با استفاده از الگوریتم ساده تشخیص چهره داده ها را به صورت زیر نرمال سازی و ذخیره میکنیم:

```
#include <opencv2/opencv.hpp>
#include <opencv2/dnn.hpp>
#include <iostream>
#include <filesystem>
#include <string>
#include <regex>

using namespace std;
using namespace cv;
namespace fs = std::filesystem;

int main() {
    // برای تشخیص چهره DNN مدل
    string faceProto = "face_detector/opencv_face_detector.pbtxt";
    string faceModel = "face_detector/opencv_face_detector_uint8.pb";

    // بارگذاری مدل DNN
    dnn::Net faceNet = dnn::readNetFromTensorflow(faceModel, faceProto);

    // پوشه های تصاویر ورودی
    vector<string> partFolders = {"part1", "part2", "part3"};

    // پوشه برای ذخیره چهره های برش خورده
    string outputFolder = "trainImages";
    if (!fs::exists(outputFolder)) {
        fs::create_directory(outputFolder);
    }

    // حلقه برای خواندن تصاویر از پوشه های مختلف
    for (const auto& folder : partFolders) {
        for (const auto& entry : fs::directory_iterator(folder)) {
            if (entry.is_regular_file()) {
                string imagePath = entry.path().string();

                // خواندن تصویر
                Mat frame = imread(imagePath);
                if (frame.empty()) {
                    cerr << "Error loading image: " << imagePath << endl;
                    continue;
                }
            }
        }
    }
}
```

```

        // برای پردازش با blob تبدیل تصویر به DNN
        Mat blob = dnn::blobFromImage(frame, 1.0, Size(300, 300),
Scalar(104.0, 177.0, 123.0), true, false);
        faceNet.setInput(blob);
        Mat detections = faceNet.forward();
        Mat detectionMat(detections.size[2], detections.size[3], CV_32F,
detections.ptr<float>());

        // پردازش هر چهره شناسایی شده
        for (int i = 0; i < detectionMat.rows; i++) {
            float confidence = detectionMat.at<float>(i, 2);
            if (confidence > 0.5) { // آستانه اطمینان
                // مختصات چهره شناسایی شده
                int x1 = static_cast<int>(detectionMat.at<float>(i, 3) *
frame.cols);
                int y1 = static_cast<int>(detectionMat.at<float>(i, 4) *
frame.rows);
                int x2 = static_cast<int>(detectionMat.at<float>(i, 5) *
frame.cols);
                int y2 = static_cast<int>(detectionMat.at<float>(i, 6) *
frame.rows);

                x1 = max(0, x1);
                y1 = max(0, y1);
                x2 = min(frame.cols - 1, x2);
                y2 = min(frame.rows - 1, y2);

                Rect faceBox(Point(x1, y1), Point(x2, y2));

                // برش چهره از تصویر
                Mat face = frame(faceBox);

                string fileName = entry.path().filename().string();

                // ذخیره چهره برش خورده با همان نام اصلی
                string outputFilePath = outputFolder + "/" + fileName;
                imwrite(outputFilePath, face);
            }
        }
    }
}

return 0;
}

```



شکل 6-3-1: دیتاست شامل طیف گسترده ای از چهره ها به همراه لیبل می باشد

6-3-2- آموزش مدل با کتابخانه Tensorflow در Python

TensorFlow یک کتابخانه متن باز است که توسط Google برای توسعه و آموزش مدل های یادگیری ماشین و یادگیری عمیق طراحی شده است. این کتابخانه به طور خاص برای پردازش داده های بزرگ و پیچیده مانند تصاویر، ویدئوها، و داده های صوتی ساخته شده است. TensorFlow به برنامه نویسان و محققان این امکان را می دهد که مدل های یادگیری ماشین را با استفاده از گراف های محاسباتی بسازند، که در آن ها هر گره نمایانگر یک عملیات ریاضی و هر یال نمایانگر داده ها است.

TensorFlow C++ API به عنوان یک رابط برنامه نویسی برای استفاده از کتابخانه TensorFlow در زبان C++ طراحی شده است. این API به توسعه دهندگان این امکان را می دهد که مدل های یادگیری ماشین را مستقیماً در برنامه های C++ پیاده سازی، آموزش و استقرار دهند، بدون نیاز به استفاده از Python. TensorFlow C++ API تمامی قابلیت های اصلی مانند ساخت، آموزش، و استقرار مدل های یادگیری عمیق را فراهم می کند، به علاوه اینکه از محاسبات موازی و بهینه سازی های سطح پایین بهره می برد تا عملکرد بالاتری ارائه دهد. این API به ویژه برای استفاده در سیستم های با محدودیت منابع مناسب است، جایی که نیاز به استفاده از زبان C++ به دلیل کارایی بالاتر و دسترسی مستقیم به منابع سخت افزاری وجود دارد.

مدل شبکه عصبی را با دیتاست نرمال سازی شده آموزش می دهیم:

```

import os
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing import image
from tensorflow.keras.utils import to_categorical
import re

# پوشه تصاویر
train_images_dir = 'trainImages' # مسیر پوشه تصاویر

# پارامترهای پیش‌پردازش
img_height = 224
img_width = 224

# لیست فایل‌ها در پوشه
image_files = os.listdir(train_images_dir)

# لیست برای ذخیره داده‌ها و برچسب‌ها
X_train = []
y_age = []
y_gender = []
y_race = []

# بارگذاری تصاویر و استخراج برچسب‌ها
for img_file in image_files:
    # استخراج برچسب‌ها از نام فایل
    # است [age]_[gender]_[race]_[datetime].jpg فرض می‌کنیم که نام فایل به شکل
    match = re.match(r"(\d+)_(\d+)_(\d+)\d+.jpg", img_file)

    if match:
        age = int(match.group(1))
        gender = int(match.group(2))
        race = int(match.group(3))

        # بارگذاری تصویر
        img_path = os.path.join(train_images_dir, img_file)
        img = image.load_img(img_path, target_size=(img_height, img_width))

        # تبدیل تصویر به آرایه numpy
        img_array = image.img_to_array(img)

        # نرمال‌سازی تصویر به بازه [0, 1]
        img_array = img_array / 255.0

        # اضافه کردن تصاویر به داده‌های ورودی

```

```

X_train.append(img_array)

# اضافه کردن پرچسب‌ها
y_age.append(age)
y_gender.append(gender)
y_race.append(race)

# تبدیل داده‌ها به آرایه‌های numpy
X_train = np.array(X_train)
y_age = np.array(y_age)
y_gender = np.array(y_gender)
y_race = np.array(y_race)

# تبدیل پرچسب‌ها به categorical
y_gender = to_categorical(y_gender, num_classes=2) # 0 (male) و 1 (female)
y_race = to_categorical(y_race, num_classes=5) # 0 (White), 1 (Black), 2 (Asian), 3 (Indian), 4 (Others)

# برای دسته‌بندی سن، می‌توانید سن‌ها را به دسته‌های مختلف تقسیم کنید (مثلاً 0-18، 19-30، 31-45 و غیره)
# به عنوان مثال، برای ایجاد 5 دسته سن:
age_bins = [0, 18, 30, 45, 60, 116]
y_age = np.digitize(y_age, bins=age_bins) # تقسیم سن‌ها به دسته‌ها
y_age = to_categorical(y_age, num_classes=len(age_bins)-1) # categorical تبدیل به

# ساخت مدل CNN
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu',
input_shape=(img_height, img_width, 3)),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(128, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),

    # خروجی جنسیت
    tf.keras.layers.Dense(2, activation='softmax', name="gender_output"),

    # خروجی سن
    tf.keras.layers.Dense(5, activation='softmax', name="age_output"), # دسته 5
    سن

    # خروجی نژاد

```

```

        tf.keras.layers.Dense(5, activation='softmax', name="race_output") # دسته
نژاد
    ])

# کامپایل مدل
model.compile(optimizer='adam',
               loss={'gender_output': 'categorical_crossentropy',
                    'age_output': 'categorical_crossentropy',
                    'race_output': 'categorical_crossentropy'},
               metrics=['accuracy'])

# آموزش مدل
model.fit(X_train,
         {'gender_output': y_gender, 'age_output': y_age, 'race_output':
y_race},
         epochs=10,
         batch_size=32)

model.save('gender_age_race_model.h5')

```

مدل CNN طراحی شده به طور خاص برای شناسایی ویژگی‌های مختلف در داده‌های تصویری به کار می‌رود. شبکه‌های عصبی کانولوشنی به دلیل قابلیت‌های بالای خود در استخراج ویژگی‌ها از تصاویر و داده‌های متوالی (مانند ویدیو یا صدا) بسیار محبوب هستند.

لایه‌های کانولوشن (Convolutional Layers)

- **Conv2D(32, (3, 3), activation='relu')**: اولین لایه کانولوشن که از 32 فیلتر با ابعاد 3×3 برای شناسایی ویژگی‌های اولیه تصویر (مانند لبه‌ها، خطوط و بافت‌ها) استفاده می‌کند. این لایه به شبکه کمک می‌کند تا ویژگی‌های ساده‌ای مانند لبه‌ها یا رنگ‌ها را از تصاویر استخراج کند.
- **Conv2D(64, (3, 3), activation='relu')**: لایه کانولوشن دوم که تعداد فیلترها را به 64 افزایش می‌دهد، به منظور شناسایی ویژگی‌های پیچیده‌تر در تصویر مانند ترکیب‌های لبه‌ها و بافت‌ها.
- **Conv2D(128, (3, 3), activation='relu')**: لایه کانولوشن سوم که از 128 فیلتر استفاده می‌کند، برای شناسایی ویژگی‌های حتی پیچیده‌تر و الگوهای سطح بالاتر در تصویر طراحی شده است.

لایه‌های ماکس پولینگ (MaxPooling Layers)

- **MaxPooling2D((2,2))**: لایه‌های ماکس پولینگ بعد از هر لایه کانولوشن قرار دارند و وظیفه آن‌ها کاهش ابعاد ویژگی‌های استخراج‌شده و افزایش ویژگی‌های معنایی آن‌هاست. این لایه‌ها به مدل کمک می‌کنند که ویژگی‌های مهم تصویر را حفظ کرده و محاسبات را ساده‌تر کنند.

لایه‌های فلت (Flatten Layer)

- **Flatten()**: پس از اینکه ویژگی‌های تصویر با استفاده از لایه‌های کانولوشن و پولینگ استخراج شدند، لایه فلت این ویژگی‌ها را به آرایه یک‌بعدی تبدیل می‌کند تا به ورودی لایه‌های Dense منتقل شود. این لایه به مدل این امکان را می‌دهد که ویژگی‌های پیچیده را به ورودی‌های مناسب پیش‌بینی تبدیل کند.

لایه‌های Dense (Fully Connected Layers)

- **Dense(128, activation='relu')**: این لایه یک لایه کاملاً متصل است که به شبکه کمک می‌کند ویژگی‌های استخراج‌شده را ترکیب کرده و تصمیمات نهایی را در مورد پیش‌بینی‌ها بگیرد. از تابع فعال‌سازی relu برای فعال کردن نوروں‌ها استفاده می‌شود که باعث می‌شود مدل توانایی یادگیری ویژگی‌های پیچیده‌تر را داشته باشد.

این مدل به‌طور همزمان سه پیش‌بینی مختلف را انجام می‌دهد: پیش‌بینی جنسیت، سن و نژاد. به این منظور، سه لایه خروجی مختلف در نظر گرفته شده است:

لایه خروجی جنسیت

- **Dense(2, activation='softmax', name="gender_output")**: این لایه برای پیش‌بینی جنسیت طراحی شده است. از تابع فعال‌سازی softmax استفاده می‌شود که به‌طور خاص برای مسائل دسته‌بندی چندکلاسه مناسب است. در اینجا دو کلاس داریم: مرد (0) و زن (1). تابع softmax احتمال هر کدام از این دو کلاس را محاسبه می‌کند.

لایه خروجی سن

- **Dense(5, activation='softmax', name="age_output")**: این لایه برای پیش‌بینی دسته‌بندی سنی استفاده می‌شود. داده‌ها به پنج دسته سنی تقسیم شده‌اند (مثلاً 0-18, 19-30, 31-45 و غیره). از تابع softmax برای پیش‌بینی احتمال تعلق به هر دسته سنی استفاده می‌شود.

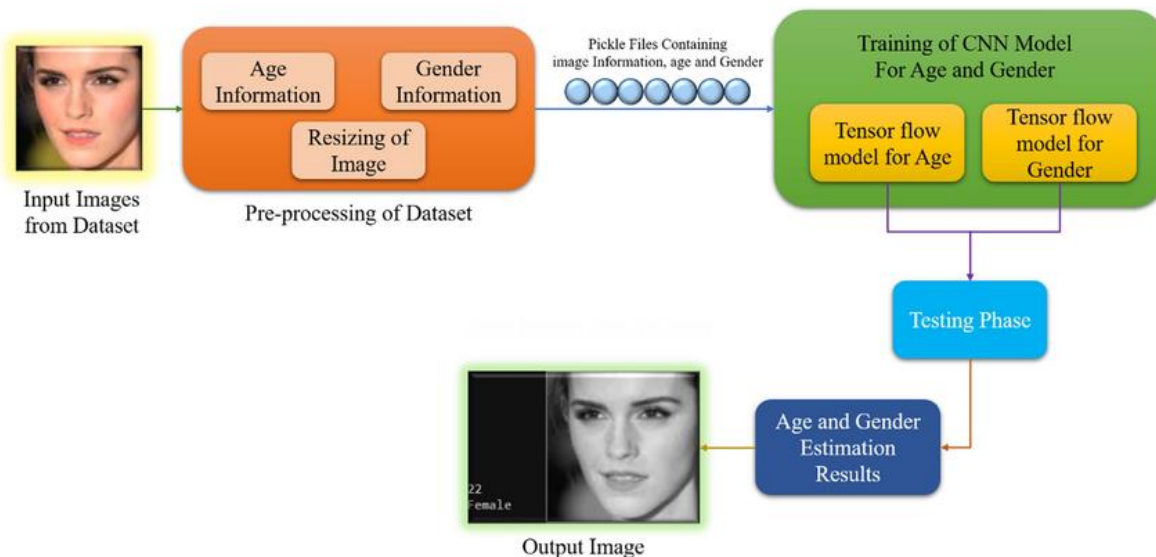
لایه خروجی نژاد

- `Dense(5, activation='softmax', name="race_output")`: این لایه برای پیش‌بینی نژاد طراحی شده است. پنج کلاس مختلف نژادی (مانند سفید، سیاه، آسیایی، هندی و دیگران) در نظر گرفته شده‌اند. مانند بقیه لایه‌ها، از تابع `softmax` برای دسته‌بندی داده‌ها به این پنج کلاس استفاده می‌شود.

کامپایل مدل

- `optimizer='adam'`: در این مدل از بهینه‌ساز Adam استفاده می‌شود که یکی از موثرترین بهینه‌سازها برای شبکه‌های عصبی است. این بهینه‌ساز به‌طور خودکار نرخ یادگیری را تنظیم می‌کند.
- `loss='categorical_crossentropy'`: برای هر خروجی از این تابع هزینه استفاده می‌شود. `categorical_crossentropy` به‌طور معمول برای مسائل دسته‌بندی چندکلاسه مناسب است.
- `metrics=['accuracy']`: به‌عنوان معیار ارزیابی عملکرد آن در طول آموزش و تست استفاده می‌شود.

مدل با استفاده از داده‌های آموزشی (X_{train}) و برچسب‌های مربوط به جنسیت، سن و نژاد (y_{gender} , y_{age} , y_{race}) آموزش داده می‌شود. برای هر دسته‌بندی، مدل به‌طور جداگانه خطا را محاسبه کرده و از طریق فرآیند بهینه‌سازی، وزن‌های مدل را به‌گونه‌ای تنظیم می‌کند که دقت پیش‌بینی‌ها افزایش یابد.



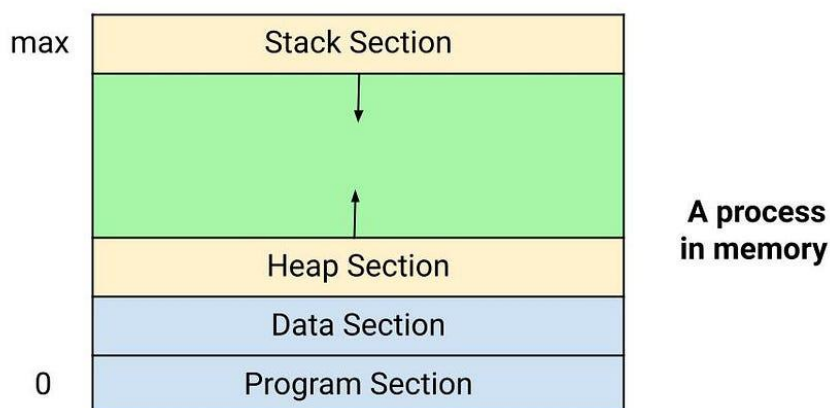
شکل 6-3-2: آموزش مدل CNN با اعمال پیش پردازش و نرمالسازی بر روی دیتاست

فصل هفتم

پخش فایل ویدئویی و صوتی

7-1- فرآیندها در سیستم عامل

فرآیندها در سیستم عاملها واحدهای اجرایی هستند که به برنامه‌های در حال اجرا اشاره دارند. هر فرآیند شامل یک مجموعه از دستورات است که به‌طور پیوسته توسط واحد پردازش مرکزی (CPU) اجرا می‌شوند. علاوه بر کد اجرایی، فرآیند شامل اطلاعات دیگری نظیر داده‌ها، فایل‌های باز، وضعیت پردازنده، و اطلاعات کنترلی (مانند شمارنده برنامه و رجیسترها) است. فرآیندها به دو نوع تقسیم می‌شوند: فرآیندهای در حال اجرا (Running) که توسط پردازنده پردازش می‌شوند و فرآیندهای در انتظار (Blocked) که منتظر منابع یا ورودی/خروجی هستند. مدیریت فرآیندها در سیستم عاملها شامل ایجاد، نابودسازی، و تغییر وضعیت فرآیندها است. سیستم عامل با استفاده از الگوریتم‌های زمان‌بندی مانند Round Robin یا First-Come-First-Served، تخصیص منابع به فرآیندها را انجام می‌دهد. همچنین، فرآیندها می‌توانند از طریق مفهوم پردازش‌های چندگانه (Multitasking) یا چند پردازشی (Multiprocessing) به‌طور همزمان اجرا شوند، که این قابلیت به بهینه‌سازی استفاده از منابع سیستم کمک می‌کند.



شکل 7-1: یک فرآیند در حافظه

7-2- نحوه پخش فایل های ویدئویی و صوتی با مدیریت فرآیند ها

برای اینکه در مورد چگونگی پخش فایل های ویدئویی و صوتی در ربات تعاملی سلام صحبت کنیم، ابتدا باید ساختار کلاس Monitor را بررسی کنیم. در هدر کلاس Monitor نیز مانند کلاس Camera یک Task و یک Mode تعریف شده است:

```
enum Task {
    SC_SAVER,
    READY2PLAY,
    PLAY,
    STOP,
};

enum Mode {
    VIDEO,
    AUDIO,
    GIF,
};
```

برای اینکه با نحوه استفاده از enum Task آشنا شویم باید دو متد مهم کلاس Monitor را بررسی کنیم:

```
void Monitor::setTask(Task task) {
    cout << "Monitor: new task is " << task << " | current task is " <<
currentTask << endl;
    if (task == Task::PLAY) {
        if(this->currentMode == VIDEO) {
            cout << "MONITOR: task is play: video gif" << endl;
            stopPlayback(Mode::GIF);
        }
        std::this_thread::sleep_for(std::chrono::seconds(INTERSTREAM_SLEEP_TIME));
    }
    startPlayback();
} else if (task == Task::STOP) {
    stopPlayback(this->currentMode);

    if(this->currentMode == VIDEO) {
        cout << "MONITOR: task is stop: video" << endl;
        this->currentMode = GIF;
        std::this_thread::sleep_for(std::chrono::seconds(INTERSTREAM_SLEEP_TIME));
    }
    startPlayback();
}

return;
```

```

    }
    else if (task == Task::SC_SAVER && currentTask != Task::SC_SAVER) {
        this->currentMode = Mode::GIF;
        std::this_thread::sleep_for(std::chrono::seconds(INTERSTREAM_SLEEP_TIME))
    }
    ;

    startPlayback();
}
else if(task == Task::READY2PLAY && currentTask == Task::SC_SAVER) {
    stopPlayback(Mode::GIF);
}
currentTask = task;
}

void Monitor::update() {
    switch (currentTask) {
        case Task::SC_SAVER:
            break;
        case Task::READY2PLAY:
            if (playerPids[Mode::GIF] != -1) {
                stopPlayback(Mode::GIF);
            }
            break;
        case Task::PLAY:
            if(getStreamFinished()) {
                currentTask = Task::STOP;
            }
            break;
        case Task::STOP:
            stopPlayback(this->currentMode);
            currentTask = Task::SC_SAVER;
            break;
    }
}
}

```

در این دو متد، Task برای مدیریت وضعیت‌های مختلف پخش و تعاملات مختلف استفاده می‌شود. هر وضعیت جدید (Task::READY2PLAY, Task::PLAY, Task::STOP, Task::SC_SAVER) باعث می‌شود که اقدامات مختلفی انجام شود، مانند شروع یا توقف پخش، تغییر حالت بین VIDEO و GIF، و بازگشت به حالت صرفه‌جویی در صفحه. متد setTask برای تنظیم وضعیت و انجام عملیات مرتبط با آن وضعیت استفاده می‌شود، در حالی که update وضعیت فعلی را بررسی کرده و اقدامات مناسب را برای هر وضعیت انجام می‌دهد.

برای آشنا شدن با مفهوم enum Mode باید دو متد دیگر کلاس Monitor را بررسی کنیم:

```
void Monitor::startPlayback() {

    string command;
    if(this->currentMode == VIDEO) {
        command = "mpv --fullscreen --geometry=100%:100% \"$(find " + this->currentStream + " -name \"*.mp4\" | head -n 1)\" & echo $! > mpv_pid.txt";
        cout << command << endl;
        system(command.c_str());

        FILE* pidFile = fopen("mpv_pid.txt", "r");
        if (pidFile) {
            char buffer[128];
            if (fgets(buffer, sizeof(buffer), pidFile) != nullptr) {
                try {
                    playerPid = stoi(buffer); // از فایل PID استخراج
                } catch (...) {
                    playerPid = -1;
                }
            } else {
                playerPid = -1;
            }
            fclose(pidFile);
        } else {
            playerPid = -1;
        }
        playerPids[Mode::VIDEO] = playerPid; // PID ذخیره
        streamFinished = false;
        streamStartTime = millis();

    } else if(this->currentMode == AUDIO) {
        command = "ffplay -nodisp -loglevel quiet \"" + this->currentStream + "\" & echo $!";

        FILE* pipe = popen(command.c_str(), "r");
        if (!pipe) {
            streamFinished = true;
            return;
        }

        char buffer[128];
        if (fgets(buffer, sizeof(buffer), pipe) != nullptr) {
            try {
                playerPid = stoi(buffer);
            } catch (...) {
                playerPid = -1;
            }
        }
    }
}
```

```

        } catch (...) {
            playerPid = -1;
        }
    } else {
        playerPid = -1;
    }
    playerPids[this->currentMode] = playerPid;
    pclose(pipe);
    streamFinished = false;
    streamStartTime = millis();
} else if(this->currentMode == GIF){
    std::string command = "mpv --loop --fs --volume=70 " + this->scSaverGif +
" & echo $! > gif_pid.txt";
    system(command.c_str());

    FILE* pidFile = fopen("gif_pid.txt", "r");
    if (pidFile) {
        char buffer[128];
        if (fgets(buffer, sizeof(buffer), pidFile) != nullptr) {
            try {
                playerPid = stoi(buffer); // از فایل PID استخراج
            } catch (...) {
                playerPid = -1;
            }
        } else {
            playerPid = -1;
        }
        fclose(pidFile);
    } else {
        playerPid = -1;
    }
    playerPids[Mode::GIF] = playerPid; // PID ذخیره
    streamFinished = false;
    streamStartTime = millis();
}
}

void Monitor::stopPlayback(Mode mode) {
    pid_t pp = playerPids[mode];
    if (pp > 0) {
        kill(pp, SIGTERM);
        pp = -1;
    }
    streamFinished = true;
}

```

در این دو متد، Mode برای تعیین حالت‌های مختلف پخش (ویدئو، صوتی، و GIF) استفاده می‌شود. در startPlayback، بسته به مقدار currentMode، دستورات مختلفی برای پخش انواع رسانه‌ها اجرا می‌شود و PID آن‌ها ذخیره می‌شود. در stopPlayback، با استفاده از Mode و playerPids، پخش متوقف می‌شود. Mode به این صورت به مدیریت و تفکیک حالات مختلف پخش و اجرای عملیات مرتبط کمک می‌کند.

در هر دو متد startPlayback و stopPlayback، فرآیندها نقشی کلیدی در شروع و توقف پخش رسانه‌ها دارند. در متد startPlayback، فرآیندهای جدید برای پخش ویدئو، صوت و GIF ایجاد می‌شوند و PID آن‌ها ذخیره می‌شود تا در آینده قابل شناسایی و مدیریت باشند. در متد stopPlayback، از PID برای متوقف کردن فرآیندهای پخش استفاده می‌شود. به این ترتیب، این دو متد به‌طور مستقیم با فرآیندهای سیستم برای مدیریت پخش رسانه‌ها و توقف آن‌ها ارتباط دارند.

دستورات به command prompt

در متد startPlayback، برای پخش هر نوع رسانه (ویدئو، صوتی یا GIF)، از دستوراتی استفاده می‌شود که در واقع فرآیندهای جدید را در سیستم ایجاد می‌کنند. به طور خاص:

- **فرآیند ویدئو (VIDEO):** دستور mpv با استفاده از system(command.c_str()) اجرا می‌شود. این دستور یک فرآیند جدید در سیستم برای پخش ویدئو ایجاد می‌کند. پس از اجرای دستور، PID (شناسه فرآیند) این فرآیند جدید از فایل mpv_pid.txt استخراج می‌شود. این PID در آرایه playerPids[Mode::VIDEO] ذخیره می‌شود تا به‌طور خاص فرآیند پخش ویدئو شناسایی شود. از این PID برای مدیریت یا متوقف کردن فرآیند در آینده استفاده خواهد شد.

```
mpv --fullscreen --geometry=100%:100% \"$(find " + this->currentStream + " -  
name \"*.mp4\" | head -n 1)\" & echo $! > mpv_pid.txt
```

- **فرآیند صوتی (AUDIO):** مشابه حالت ویدئو، دستور ffmpeg برای پخش فایل صوتی اجرا می‌شود. مجدداً، یک فرآیند جدید برای پخش صوتی ایجاد می‌شود و PID آن استخراج شده و در آرایه playerPids[Mode::AUDIO] ذخیره می‌شود.

```
ffmpeg -nodisp -loglevel quiet \"\" + this->currentStream + "\"" & echo $!
```

- **فرآیند GIF:** در این حالت، دستور mpv برای پخش یک فایل GIF با تنظیمات خاص (loop و fullscreen) اجرا می‌شود. این هم منجر به ایجاد یک فرآیند جدید می‌شود که PID آن در فایل gif_pid.txt ذخیره می‌شود. این فرآیند در آرایه playerPids[Mode::GIF] ذخیره می‌شود.

```
mpv --loop --fs --volume=70 " + this->scSaverGif + " & echo $! > gif_pid.txt
```

7-3- دسترسی به فایل های ویدئویی و صوتی با داده های کلاس Camera

در constructor کلاس Monitor سه آرگومان ورودی داده می شوند:

```
Monitor monitor("/home/salam-robo/test/assemble/videos", "/home/salam-robo/test/assemble/audios", "/home/salam-robo/test/assemble/gif3.mp4");
```

ورودی های کلاس Monitor به ترتیب مسیر دسترسی به فایل های ویدئویی، صوتی و گیف هستند.

اما در صورت تشخیص رنگ یک لباس یا تشخیص مشخصات ظاهری یک چهره، چگونه باید فایل صوتی مربوط به آن رنگ یا فایل ویدئویی مربوط به آن مشخصات ظاهری را در ادامه مسیر های تعیین شده پیدا کرد؟

در فصل چهارم کلاس FSM را بررسی کردیم. ورودی های متد update در کلاس FSM نیز نمونه های کلاس های Camera، Monitor و Sonar2 بودند. بخش هایی از آن را مرور می کنیم:

بخش مربوط به پخش فایل صوتی

```
string cd = monitor.getAudiosDirectory() + '/' + camera.getClothColor() + "/" + camera.getClothColor() + ".mp3";  
monitor.setMode(Monitor::Mode::AUDIO);  
monitor.setStream(cd);  
monitor.setTask(Monitor::Task::PLAY);
```

بخش مربوط به پخش فایل ویدئویی

```
string cd = monitor.getVideosDirectory() + '/' + camera.getFaceFeatures().gender + '/' + camera.getFaceFeatures().age;  
monitor.setStream(cd);  
monitor.setMode(Monitor::Mode::VIDEO);  
monitor.setTask(Monitor::Task::PLAY);
```

همانطور که در کد های بالا مشخص است، با استفاده از داده های کلاس Camera که با متد update در اختیار کلاس FSM قرار گرفته است، مسیر دسترسی به فایل های مورد نظر را با استفاده از متد setStream به کلاس Monitor اطلاع می دهیم تا در متغیر currentStream آن را ذخیره کند و در دستورات سیستم عاملی به command prompt فایل مورد نظر انتخاب شود.

- [1] <https://github.com/alikashipazha/salam-robo/tree/main/assemble>
- [2] <https://github.com/smahesh29/Gender-and-Age-Detection>
- [3] <https://susanqq.github.io/UTKFace>